

Prometheus: A Heterogeneous Collaborative System for High-Throughput Encrypted Traffic Analysis

Chao Liu*
Tsinghua University
Shenzhen, China
chao-
liu25@mails.tsinghua.edu.cn

Zhuochen Fan†
Pengcheng Laboratory
Shenzhen, China
fanzc@pku.edu.cn
fanzhch@pcl.ac.cn

Dong Wen
National University of
Defense Technology
Changsha, China
wendong19@nudt.edu.cn

Gareth Tyson
Hong Kong University of
Science and Technology
(Guangzhou)
Guangzhou, China
gtyson@ust.hk

Peiyuan Zong
Pengcheng Laboratory
Shenzhen, China
zongpy@pcl.ac.cn

Zeyu Luan
Pengcheng Laboratory
Shenzhen, China
luanzy@pcl.ac.cn

Guorui Xie
Pengcheng Laboratory
Shenzhen, China
xiegr@pcl.ac.cn

Yong Jiang*
Tsinghua University
Shenzhen, China
jiangy@sz.tsinghua.edu.cn

Mingwei Xu
Tsinghua University
Beijing, China
xumw@tsinghua.edu.cn

Qing Li†
Pengcheng Laboratory
Shenzhen, China
liq@pcl.ac.cn

Abstract

High-speed encrypted traffic analysis must balance semantic accuracy with line-rate processing on 10–100 Gbps links. Existing classifiers choose between expensive deep models, which improve robustness under protocol evolution and traffic drift, and lightweight models, which provide throughput but degrade on long-tail traffic. We present Prometheus, a heterogeneous collaborative system that separates real-time data-plane classification from backend semantic analysis. A Light Model handles common flows on the fast path, while a Heavy Model analyzes low-confidence flows and supplies online distillation signals. Confidence-based routing, asynchronous evolution and a Light Model keep adaptation off the data path. Offline, the Heavy Model averages 94.0% accuracy; online, the collaborative runtime averages 93.1% accuracy, 5.25 points above Light-only. On a 10 Gbps DPDK testbed, the full system reaches 9.8 Gbps with just 2.1 ms end-to-end latency, while the Light Model alone sustains line rate with 0.3 ms latency. On a 100 Gbps-capable multi-GPU testbed, the Light Model scales to 44.7 Gbps and exposes NIC/host/GPU

data movement, rather than GPU inference alone, as the dominant scaling bottleneck.

CCS Concepts: • Networks → Network measurement; Network performance analysis; Network monitoring.

Keywords: Encrypted Traffic Analysis, High-Throughput Systems, Online Distillation, GPU Inference

ACM Reference Format:

Chao Liu, Zhuochen Fan, Dong Wen, Gareth Tyson, Peiyuan Zong, Zeyu Luan, Guorui Xie, Yong Jiang, Mingwei Xu, and Qing Li. 2027. Prometheus: A Heterogeneous Collaborative System for High-Throughput Encrypted Traffic Analysis. In *22nd European Conference on Computer Systems (EuroSys '27)*, April 19–23, 2027, Rabat, Morocco. ACM, New York, NY, USA, 24 pages. <https://doi.org/10.1145/3842654.3848566>

1 Introduction

Modern networks increasingly rely on TLS, QUIC, and related encryption protocols to protect confidentiality [29, 44]. This shift removes payload visibility from middleboxes and monitoring systems, forcing encrypted traffic analysis to infer application and service semantics from observable side channels such as packet sizes, directions, timing, and transport metadata [25, 46]. The resulting problem is no longer only a modeling problem. At production speeds, an encrypted-traffic classifier must be accurate under fast protocol evolution, but it must also process traffic at line rate with predictable latency and bounded GPU demand. This requirement is becoming more stringent as monitoring points move from 10 Gbps enterprise links toward 100 Gbps edge and backbone deployments, where per-flow deep inference

*Chao Liu and Yong Jiang are with Tsinghua Shenzhen International Graduate School, Tsinghua University, and also with Pengcheng Laboratory.

†Corresponding Authors: Zhuochen Fan and Qing Li.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2971-3/2027/04

<https://doi.org/10.1145/3842654.3848566>

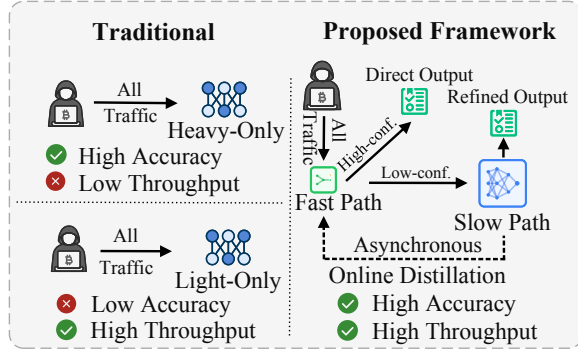


Figure 1. Comparison between traditional encrypted-traffic classification pipelines and Prometheus.

can quickly turn model cost, CPU preprocessing, and host-to-GPU transfer into system bottlenecks.

This requirement creates a deployment dilemma. High-capacity models such as Transformers can capture fine-grained protocol behavior and improve accuracy on difficult or drifting traffic, but applying them uniformly to every flow is too expensive for a real-time network appliance, and becomes especially untenable at 100 Gbps-scale input rates. Lightweight models offer attractive throughput, but their limited capacity makes them brittle on long-tail services, encryption-variant shifts, and sudden changes in traffic mix. In operational terms, the system is squeezed between semantic depth and data-plane throughput: robustness asks for a heavy model, while high-speed processing asks for a light model whose surrounding data path can also scale.

Existing encrypted-traffic classification systems usually resolve this tension via a static design point. Statistical and sequence-based methods are efficient but depend on shallow features and often require complete flows [38, 51, 53]. Deep learning and pre-training methods improve representation quality [8, 24, 34, 75, 78], but their inference costs make them difficult to deploy directly in high-speed settings. More importantly, a single uniform pipeline cannot exploit the asymmetry of real traffic: most flows are easy and repetitive, while only a small fraction requires expensive semantic inspection.

We propose **Prometheus**, a heterogeneous collaborative system for high-throughput encrypted traffic analysis. As shown in Fig. 1, Prometheus decouples real-time inference from deep semantic analysis. A Light Model runs on the fast path and processes the majority of flows with low latency. In parallel, a Heavy Model runs as a backend semantic engine for low-confidence flows, where it provides more accurate predictions and generates distillation targets. An online evolution loop updates the Light Model asynchronously, allowing the data plane to adapt to concept drift without blocking packet processing.

This architecture turns robustness from a per-flow cost into a scheduled system resource. Confidence-based routing concentrates Heavy Model computation on flows where

it is most useful. Asynchronous distillation amortizes the cost of adaptation over batches of difficult samples, and atomic model switching confines update stalls to a short control-plane event. Protocol-aware attention lets the Heavy Model reason over encrypted side-channel structure, including temporal locality, packet-size regularity, directionality, and request-response causality, while the Light Model uses compact CNN and statistical streams for efficient inference.

Our evaluation shows that Prometheus provides both classification quality and deployable performance, including a dedicated scaling study on a 100 Gbps-capable platform. Across diverse TLS 1.3 and mixed-cipher datasets, Prometheus maintains over 92% accuracy across encryption variants. In a real-world 10 Gbps testbed, the collaborative system achieves 9.8 Gbps throughput with 2.1 ms latency, and the Light-only fast path reaches line rate with 0.3 ms latency. On the 100 Gbps-capable multi-GPU platform, the Light Model reaches 18.5 Gbps on one GPU and 44.7 Gbps on four GPUs, while the original Heavy Model remains below 3 Gbps even with four GPUs. The breakdown shows that NIC-to-host transfer and host-to-GPU movement, rather than GPU inference alone, dominate the remaining latency. The Light Model handles over 85% of flows without invoking the Heavy Model, while online distillation restores accuracy under drift.

This paper makes the following contributions:

- We present Prometheus, a heterogeneous heavy-light system architecture that decouples line-rate encrypted traffic classification from expensive protocol-semantic analysis.
- We design confidence-driven routing, asynchronous online distillation, and atomic Light Model installation to allocate GPU resources to difficult flows while keeping the fast path non-blocking.
- We introduce protocol-aware attention and head-aligned pre-training to capture encrypted traffic structure from observable metadata.
- We implement and evaluate Prometheus on a real 10 Gbps testbed and a 100 Gbps-capable multi-GPU platform, demonstrating line-rate 10 Gbps deployment, 44.7 Gbps Light Model scaling, and a bottleneck analysis showing that data movement becomes the dominant barrier on the path toward 100 Gbps traffic analysis.

2 Background and Motivation

2.1 Representations of Network Flows

Network traffic is structurally heterogeneous and multi-modal. While raw bytes offer fine-grained semantics in plain-text, encryption randomization renders byte-level features sparse and brittle. To recover implicit semantics under encryption, classifiers rely on complementary side-channel views: temporal dynamics (IAT) for application pacing, packet length and direction for protocol fingerprints, and observable transport structure for request-response dependencies.

Gap in existing research. Existing works [11, 28, 33, 34, 41, 55, 68, 75, 78] often treat these modalities in isolation or via concatenation, failing to capture the intrinsic structural priors inherent in network protocols. For instance, request-response behavior imposes causal constraints, where an uplink request shapes the expected characteristics of subsequent downlink responses. Generic fusion mechanisms treat traffic as flat data sequences, lacking the structural priors to explicitly model such cross-modal causalities. This leads to feature misalignment and limited interpretability—a gap that motivates architectures capable of embedding protocol-level knowledge directly into feature learning.

2.2 The Asymmetry of Traffic Complexity

A critical, yet often overlooked, characteristic of network traffic is the asymmetry in classification difficulty, which manifests in two dimensions:

Systemic Asymmetry. Our collaborative-inference results later show that, at $\theta = 0.5$, our Light Model handles roughly 84–85% of flows without needing to invoke the Heavy Model. This observation motivates treating expensive semantic analysis as a scarce backend resource rather than a per-flow cost. The remaining low-confidence traffic, such as rare services, encrypted tunnels, or drifted protocol behaviors, requires the high-capacity reasoning of large-scale Transformer models.

Load Asymmetry. This distribution creates a systems challenge. Existing deployments typically apply the same model to every flow, so the worst-case cost of a high-capacity classifier becomes the common-case cost of the entire appliance. Transformer models in particular have deterministic and expensive computational footprints (e.g., $O(L^2)$ complexity). When a burst contains many ambiguous or drifted flows, a uniform heavy pipeline can saturate GPU queues and cause receive-buffer buildup even if the average traffic rate remains within the nominal link capacity. Without an architectural mechanism to decouple simple screening from deep inspection, the system lacks a computational circuit breaker for load spikes.

2.3 Motivation and Design Goals

Based on the representational gaps and the systems pressure of uniform pipelines, we define three primary design goals for Prometheus:

G1: Protocol-Aware Semantic Fusion. Instead of prior-blind concatenation, the system must explicitly model the structural dependencies (e.g., request-response causality) between different traffic views to produce transferable representations across traffic mixes.

G2: Decoupled Heavy-Light Inference. To resolve the tension between representation depth and line-rate service, the architecture should decouple deep protocol inspection from high-throughput screening. This allows the system to

dynamically allocate its computational budget, routing only “hard samples” to the expensive backend.

G3: Closed-Loop Online Adaptation. To handle concept drift without requiring online ground-truth labels, the system requires a self-evolving mechanism where the heavy model acts as a dynamic teacher, distilling its knowledge back to the light model in real-time.

3 System Model and Assumptions

We target an encrypted-traffic analysis appliance deployed at a network edge or enterprise gateway. The appliance receives traffic from a high-speed NIC, extracts observable side-channel features without decrypting payloads, and classifies flows for downstream monitoring or policy enforcement. The deployment contains a fast data path for common-case inference and a GPU-backed slow path for deeper semantic analysis.

Traffic model. Encrypted traffic is heterogeneous and non-stationary. Most flows are repetitive web, update, and service traffic whose behavior can be recognized from shallow side-channel features. A smaller long-tail portion contains rare services, bursty flows, encryption-variant shifts, or protocol behaviors that require richer modeling. Traffic also arrives in bursts, so the classifier must tolerate short-term load spikes without blocking the receive path.

Performance target. The system is designed to sustain line-rate processing on a 10 Gbps deployment and to expose the scaling bottlenecks that arise on 100 Gbps-class platforms. Its primary systems goals are sustained throughput, millisecond-scale end-to-end latency, bounded heavy-model invocation rate, and stable accuracy under concept drift.

Assumptions. Prometheus observes packet sizes, directions, timing, and transport metadata such as TCP sequence and acknowledgment numbers when available. It does not inspect plaintext payloads or require application cooperation. The Heavy Model can run on a backend GPU and may update the Light Model asynchronously; the data plane continues serving traffic with the currently active Light Model while distillation is running. The routing decision is made solely from the Light Model’s own output confidence; no separate mechanism pre-classifies a flow as “heavy” or “light” before inference.

Stress conditions. We model operational stress from extreme traffic loads and distribution shifts. Examples include sudden increases in low-confidence flows, cipher-suite changes such as AES-to-ChaCha transitions. These stresses primarily affect system load, routing decisions, and adaptation frequency.

4 Prometheus Design

4.1 Overview

We propose Prometheus, an adaptive collaborative system that separates line-rate inference from deep protocol-semantic

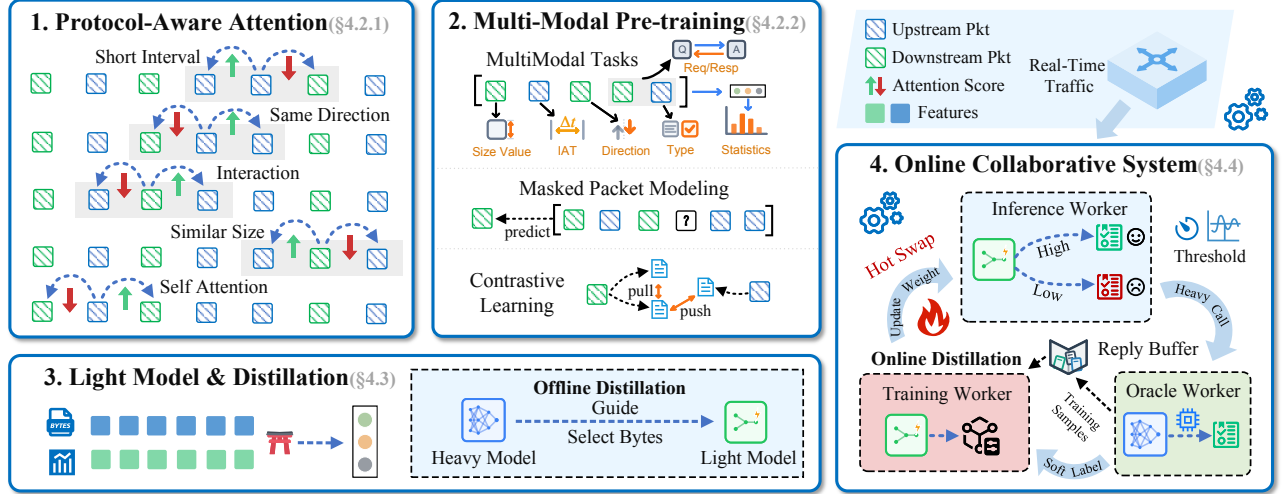


Figure 2. The framework of Prometheus.

analysis. The fast path is a resource-friendly Light Model (§4.3) that classifies the majority of flows with bounded latency. The slow path is a backend Heavy Model (§4.2) that uses protocol-guided attention to analyze ambiguous flows. As illustrated in Fig. 2, a confidence-driven scheduler (§4.4) connects the two paths: high-confidence flows are released immediately, while low-confidence flows are mirrored to the Heavy Model and buffered for online distillation. Updated Light Model weights are installed asynchronously, so adaptation does not block packet processing.

Design rationale. We choose heavy-light collaboration instead of pure hardware offload because encrypted traffic semantics evolve faster than fixed-function dataplane logic; hardware should accelerate inference and data movement, while software models preserve flexibility. This follows the broader systems lesson from edge traffic analytics that forwarding and deep analysis should be decoupled and connected asynchronously [23]. We use confidence-based routing instead of round-robin or random splitting because traffic difficulty is asymmetric: most flows are easy and should not consume Heavy Model capacity. We use online distillation instead of periodic retraining or manual labeling because concept drift can occur during deployment, and the Heavy Model can generate adaptation signals from the same low-confidence flows that stress the system.

4.2 Heavy Model: Protocol-Aware Representation Learning

The Heavy Model is designed to extract robust, protocol-aware representations from encrypted traffic that support both high-accuracy classification and effective knowledge transfer. Our design addresses two critical limitations in existing deep learning approaches: (1) **treating network flows as homogeneous sequences**, thereby overlooking intrinsic protocol structure, and (2) **relying on implicit,**

data-driven learning without leveraging domain-specific priors.

To overcome these challenges, we propose two complementary innovations. First, Protocol-Aware Multi-Head Attention (§4.2.1) injects observable protocol priors—such as temporal locality, directional asymmetry, and request-response causality—directly into the self-attention mechanism as learnable inductive biases, enabling the model to focus on semantically meaningful interaction patterns while maintaining robustness to noise. Second, Head-Aligned Multi-Task Pre-training (§4.2.2) constructs self-supervised objectives that are explicitly aligned with each attention head, ensuring that protocol priors are effectively internalized during training on unlabeled, noisy traffic. Together, these mechanisms ground the model in protocol semantics while preserving adaptability to diverse encrypted protocols.

4.2.1 Protocol-Aware Multi-Head Attention. Standard Transformer attention often yields uninterpretable, “noisy” dependencies in encrypted traffic (Fig. 3, top). To enhance interpretability, we inject protocol-level priors that mimic expert analysis. As illustrated by the “Direction Head” (Fig. 3, bottom), our approach explicitly aligns the model’s focus with intrinsic protocol structures such as directional consistency ensuring representations are grounded in protocol semantics rather than generic data-driven correlations.

Formally, given a flow representation H consisting of N packets, the attention computation of the h -th head is defined as:

$$\text{Attn}_h = \text{Softmax} \left(\frac{Q_h K_h^T}{\sqrt{d_k}} + \lambda_h \cdot B_h \right) V_h, \quad (1)$$

where Q_h and K_h denote content-based projections capturing byte-level semantics; $B_h \in \mathbb{R}^{N \times N}$ is a protocol bias matrix constructed from packet metadata; and λ_h is a learnable scaling factor that adaptively controls the strength of prior injection.

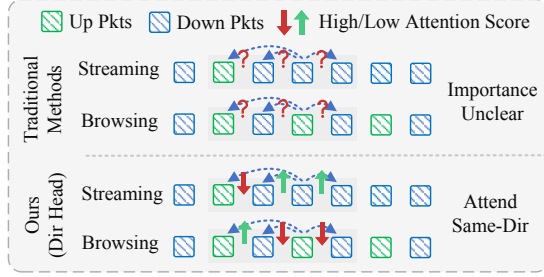


Figure 3. Motivation for the Direction Head. Traditional attention scores are often uninterpretable across different traffic types. Our Direction Head enhances feature representation by explicitly attending to same-direction packets, thereby capturing robust directional priors.

We design five parallel attention heads in total: one standard content head and four protocol-guided functional heads.

Temporal Proximity Head. Network traffic exhibits strong temporal locality, where packets belonging to the same interaction round tend to arrive in close temporal proximity. However, inter-arrival times (IATs) span a wide dynamic range, from microsecond-level ACKs to second-level keep-alive messages, and are heavily affected by network jitter. Linear positional encodings are therefore insufficient to capture such multi-scale temporal characteristics. To address this, we model temporal relationships using a logarithmic-scale Gaussian kernel. The bias between packets i and j is defined as:

$$\mathbf{B}_{i,j}^{\text{temp}} = - \left(\frac{\log_{10}(|\Delta t_{i,j}| + 1)}{\sigma_t} \right)^2, \quad (2)$$

where $\Delta t_{i,j}$ denotes the time difference in milliseconds, and σ_t is a hyperparameter controlling the temporal bandwidth of the kernel. Through logarithmic transformation, this design captures both dense traffic bursts and sparse control signals, while suppressing jitter-induced noise and encouraging the model to focus on temporally adjacent semantic groups.

Size Similarity Head. In encrypted protocols, packet size serves as a key functional fingerprint. For example, MTU-sized packets often correspond to file transfers, whereas small packets may indicate handshakes or acknowledgments. Based on the assumption that functionally similar packets tend to have similar sizes, we compute pairwise size similarity within a local window w_s :

$$\mathbf{B}_{i,j}^{\text{size}} = \mathbb{I}(|i - j| \leq w_s) \cdot \frac{\min(s_i, s_j)}{\max(s_i, s_j) + \epsilon}, \quad (3)$$

where s_i is the packet size, w_s is the window size, $\mathbb{I}(\cdot)$ is the indicator function, and ϵ is a small constant for numerical stability. This bias encourages the aggregation of packets with similar transmission characteristics, allowing the model to

cluster functionally coherent subsequences within complex flow sequences.

Directional Consistency Head. Real-world traffic often exhibits strong directional dominance (e.g., massive downstream in file downloads). This head enforces directional priors by strengthening the attention between packets traveling in the same path. As shown in Fig. 3, this mechanism prevents the model from being distracted by interleaving noise and forces it to focus on the continuous data stream within a single direction, which is critical for identifying asymmetric protocol behaviors such as media streaming or bulk content delivery. The directional bias is defined as:

$$\mathbf{B}_{i,j}^{\text{dir}} = \mathbb{I}(d_i = d_j) \cdot \mathbb{I}(|i - j| \leq w_d), \quad (4)$$

where $d_i \in \{+1, -1\}$ denotes packet direction (uplink or downlink), and w_d is the window size. This formulation explicitly promotes semantic aggregation within homogeneous directional packet sequences.

Protocol Semantics Head with Hybrid Interaction. To model request-response causality while avoiding over-reliance on TCP plaintext headers (which limits applicability to QUIC/UDP traffic) we design a *hybrid fallback mechanism*:

- **TCP-Aware Mode:** When valid TCP headers are available, precise causal matching is performed using sequence and acknowledgment numbers:

$$\mathcal{B}_{i,j}^{\text{tcp}} = \mathbb{I}(\text{Ack}_j = \text{Seq}_i + \text{Len}_i) \cdot \mathbb{I}(j > i) \quad (5)$$

- **Heuristic Fallback Mode:** For non-TCP traffic (e.g., QUIC), the mechanism falls back to direction-based inference with different confidence weights:

$$\mathcal{B}_{i,j}^{\text{fb}} = \begin{cases} \alpha_{\text{strong}} & \text{if } d_i = \text{Out}, d_j = \text{In}, j > i \\ \alpha_{\text{weak}} & \text{if } d_i = \text{In}, d_j = \text{Out}, j > i \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

This design sets $\alpha_{\text{strong}} > \alpha_{\text{weak}}$ to reflect the stronger causal determinism of outbound requests (Out $\rightarrow$ In) over inbound responses. It enables the model to exploit precise semantics when available while maintaining robust inference for fully encrypted protocols like HTTP/3.

Adaptive Head Gating as Meta-Controller. To capture category-specific protocol patterns (e.g., temporal jitter in VoIP vs. bursty bursts in HTTPS) that static attention fails to model, we propose an Adaptive Head Gating mechanism. We first derive a global representation $\mathbf{h}_g$ via average pooling over the packet sequence. A lightweight network then computes dynamic head weights $\mathbf{g}$:

$$\mathbf{g} = \sigma(W_2 \cdot \text{ReLU}(W_1 \mathbf{h}_g)), \quad (7)$$

where $W_{1,2}$ are learnable weights and σ denotes the sigmoid function. Unlike standard concatenation, the final representation is a dynamically weighted sum of heads scaled by $\mathbf{g}$.

Table 1. Alignment between Protocol-Aware Attention Heads and Pre-training Tasks.

Attention Head	Pre-training Task	Scope
Content Head	Packet Type Prediction	Packet
Temporal Head	IAT Bucket Prediction	Packet
Size Head	Log-Scale Size Regression	Packet
Directional Head	Direction Prediction	Packet
Semantics Head	Adjacency Prediction	Pair
All Heads (via $\mathbf{g}$)	Flow Statistical Reconstruction	Flow

This meta-controller prioritizes task-relevant feature channels (e.g., temporal vs. semantic) in real-time, significantly enhancing cross-scenario generalization.

4.2.2 Head-Aligned Multi-Task Pre-training. To internalize protocol priors, we align multi-task pre-training with specialized heads. Masked packet modeling captures the foundational semantic substrate for each head, which multi-modal prediction subsequently specializes into protocol-aware features. Contrastive learning then enforces robustness against environmental noise (e.g., jitter and packet loss). Following Tab. 1, the joint loss $\mathcal{L}_{\text{total}}$ is defined as:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{mpm}} \mathcal{L}_{\text{mpm}} + \lambda_{\text{mm}} \mathcal{L}_{\text{mm}} + \lambda_{\text{con}} \mathcal{L}_{\text{con}}, \quad (8)$$

where the terms correspond to masked packet modeling, head-aligned multi-modal prediction, and contrastive learning, respectively.

(1) Masked Packet Modeling (MPM). To provide a foundational semantic substrate for all attention heads, we randomly mask a subset of packets $\mathcal{M}$ and require the model to reconstruct their embeddings $\mathbf{h}_i$:

$$\mathcal{L}_{\text{mpm}} = 1 - \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \frac{\hat{\mathbf{h}}_i^\top \mathbf{h}_i}{\|\hat{\mathbf{h}}_i\| \|\mathbf{h}_i\|} \quad (9)$$

Instead of targeting a specific head, MPM forces the encoder to capture universal contextual dependencies across the entire flow. This foundational representation serves as the primary semantic carrier, which is then refined and specialized by the subsequent protocol-guided heads.

(2) Head-Aligned Multi-Modal Prediction. To enforce that each protocol-guided head learns its intended semantics, we design a set of lightweight, self-supervised prediction tasks aligned with the attention heads in Tab. 1. These tasks leverage only metadata observable under encryption and operate at packet or adjacent-pair level. Specifically, we predict packet roles for the content head, discretized IAT buckets for the temporal head, log-scaled sizes for the size head, next-hop direction for the directional head, and request-response adjacency for the semantics head (using direction and, when available, TCP Seq/Ack signals). In addition, we include a flow-level statistical reconstruction task that regularizes the

aggregated representation $\mathbf{g}$ by recovering aggregate statistics (e.g., size/IAT summaries, direction ratios). The combined multi-modal loss is

$$\mathcal{L}_{\text{mm}} = \sum_t \alpha_t \mathcal{L}_t + \alpha_{\text{stat}} \mathcal{L}_{\text{stat}}, \quad (10)$$

where weights α_t balance the contribution of each task. Due to space constraints, formal definitions of each loss term $\mathcal{L}_t$ are deferred to the supplementary material. This head-aligned supervision strengthens the semantic specialization of individual heads while maintaining robustness across diverse encrypted protocols.

(3) Contrastive Representation Learning. While MPM and multi-modal tasks focus on local protocol structures, they may overfit to incidental fluctuations. To learn robust “flow fingerprints” invariant to environmental noise (e.g., jitter or packet loss), we employ contrastive learning. We generate two augmented views ($\mathbf{g}, \mathbf{g}^+$) via cropping and temporal perturbations, then minimize the InfoNCE loss:

$$\mathcal{L}_{\text{con}} = -\log \frac{\exp(\text{sim}(\mathbf{g}, \mathbf{g}^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{g}, \mathbf{g}_j^+)/\tau)}, \quad (11)$$

where $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$ denotes the cosine similarity between representations, and τ is a temperature parameter. This joint framework ensures the model captures fine-grained semantics, protocol logic, and global invariance, bridging the semantic gap in encrypted traffic analysis.

4.3 Light Model: Line-Rate Fast-Path Inference

To enable real-time, low-latency traffic classification in high-speed network environments, we also design a lightweight classifier (Light Model) that serves as the *fast path*. The core design objective is to reproduce the discriminative capability of the Heavy Model under extremely constrained computational budgets using resource-friendly architectures.

4.3.1 Global Importance-Guided Input Selection. A critical challenge we face when designing a lightweight model is the trade-off between input dimensionality and information retention. Naive truncation (e.g., keeping only the first N bytes) can miss discriminative patterns in observable TLS and transport bytes. To address this, we employ a *Global Importance-Guided Input Selection* strategy, utilizing the pre-trained Heavy Model (Teacher) to identify the most discriminative traffic regions. We define the importance score $S_{i,j}^{(k)}$ for the j -th byte of the i -th packet in the k -th flow sample as the L_2 norm of the element-wise product of the embedding gradients and the embedding values:

$$S_{i,j}^{(k)} = \left\| \frac{\partial y_c}{\partial \mathbf{e}_{i,j}} \odot \mathbf{e}_{i,j} \right\|_2, \quad (12)$$

where y_c is the logit of the target class and $\mathbf{e}_{i,j}^{(k)}$ is the input embedding vector. By aggregating these scores across the

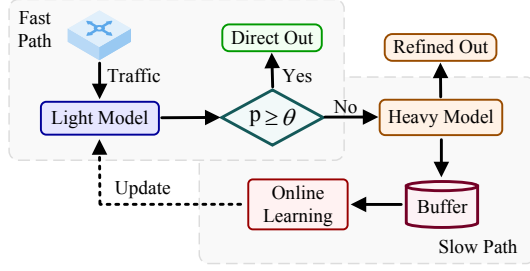


Figure 4. Workflow of Prometheus.

training set $\mathcal{D}$, we generate a global importance heatmap:

$$\mathbf{I}_{global}[i, j] = \frac{1}{N} \sum_{k=1}^N S_{i,j}^{(k)} \quad (13)$$

Based on $\mathbf{I}_{global}$, we apply a greedy selection algorithm to identify the top- K most salient packets and optimal byte windows. This yields a static global configuration that directs the Light Model’s limited receptive field to the most “average-significant” observable byte regions (e.g., TLS handshake extensions or record-layout patterns), maximizing discriminative power under strict resource constraints.

4.3.2 Architectural Design: Dual-Stream Fusion. Unlike the Heavy Model, which relies on stacked Transformer blocks, the Light Model adopts a parallel dual-stream architecture to separately capture packet-level local patterns and flow-level statistical behaviors:

Byte-level Stream: This stream leverages a compact 1D-CNN encoder to process the globally selected N bytes of each packet (as determined by the importance analysis). Compared to the $O(n^2)$ complexity of Transformers, 1D-CNNs operate in linear time $O(n)$ for extracting local protocol header patterns (e.g., TLS versions and handshake artifacts), and naturally support resource-friendly pipelined parallelization.

Statistical Stream: This branch takes packet length sequences, direction sequences, and inter-arrival times (IAT) as input. A multi-layer perceptron maps these discrete sequence features into a continuous embedding space, providing macroscopic traffic context for classification.

4.3.3 Dynamic Feature Gating. Since the contribution of different modalities varies significantly across traffic types, we introduce a *Dynamic Feature Gating* mechanism that dynamically adjusts feature weights based on global flow statistics. Let $\mathbf{v}_{byte}$ and $\mathbf{v}_{stats}$ denote the flow-level representations aggregated from the byte-stream and statistical-stream, respectively. We extract a global context vector $\mathbf{G}_{stats} \in \mathbb{R}^3$ by averaging the log-scaled packet length, direction, and log-scaled inter-arrival time (IAT) across the flow. The gating network generates normalized weights via an MLP followed by Softmax, and computes the fused flow representation as:

$$\mathbf{f}_{global} = \sum_{m \in \{\text{byte}, \text{stats}\}} \text{Softmax}(\text{MLP}_{gate}(\mathbf{G}_{stats}))_m \cdot \mathbf{v}_m, \quad (14)$$

where $\text{Softmax}(\cdot)_m$ denotes the weight assigned to modality m . This design enables the Light Model to adapt its decision strategy to each flow: increasing reliance on byte-level features for short flows with distinctive handshake or transport cues, while favoring statistical features for long flows where payload information is obfuscated but behavioral regularities persist (e.g., video streaming).

4.4 Collaborative Mechanism: Distillation & Evolution

4.4.1 Protocol-Aware Knowledge Distillation. To bridge the representational gap, we initialize the Light Model through offline supervised distillation from the Heavy Model (teacher) to the Light Model (student):

$$\mathcal{L}_{distill} = \alpha \mathcal{L}_{CE}(y, \sigma(\mathbf{z}_s)) + (1 - \alpha) \mathcal{L}_{KD}, \quad (15)$$

$$\mathcal{L}_{KD} = \tau^2 D_{KL}\left(\sigma\left(\frac{\mathbf{z}_t}{\tau}\right) \parallel \sigma\left(\frac{\mathbf{z}_s}{\tau}\right)\right), \quad (16)$$

where y is the offline ground-truth label, $\sigma(\cdot)$ is the Softmax function, τ is the temperature, and $\mathbf{z}_t, \mathbf{z}_s$ are the teacher and student logits, respectively. This process transfers complex protocol-aware decision boundaries to the student without increasing its inference latency.

4.4.2 Continuous Online Evolution. To achieve high throughput and robust accuracy in real-time scenarios, we design a confidence-aware hierarchical inference mechanism (Fig. 4). Incoming traffic is first processed by the Light Model on edge devices. Using the maximum class probability $p_{\max} = \max(\sigma(\mathbf{z}_s))$ as the confidence metric, the system routes traffic through two paths: when $p_{\max} \geq \theta$, predictions are directly output; otherwise, flows are mirrored to a backend GPU server where the Heavy Model performs deep semantic analysis. This design allows operators to balance accuracy, latency, and backend GPU load by adjusting the threshold θ .

Importantly, this routing requires no prior knowledge of which flows are difficult: every flow first traverses the Light Model, and the maximum softmax probability from that single forward pass serves as a zero-cost uncertainty proxy. We deliberately avoid more sophisticated uncertainty estimators (such as deep ensembles or Monte-Carlo dropout) because their repeated inference or additional models would add fast-path latency, thereby incompatible with 10–100 Gbps line-rate operation.

In a live deployment, the input budget is collected starting from the first packet. A flow is classified once ten packets have been collected, or earlier when the flow terminates with fewer than ten packets. High-confidence Light Model results are available immediately after this inference. A low-confidence flow has its Light Model result available first and is then mirrored to the Heavy Model, whose result is a later refinement for downstream consumers that accept asynchronous updates.

Flows routed through the slow path serve dual purposes: providing accurate immediate predictions and enabling continuous adaptation. The backend server buffers these challenging samples, and the Heavy Model generates reliable soft labels using Protocol-Aware Multi-Head Attention. Online updates use these Heavy Model soft targets as pseudo-labels; ground-truth labels are used only during offline training and evaluation. Since traffic distributions exhibit concept drift due to protocol updates or evolving obfuscation strategies, periodic model evolution is necessary.

The Heavy Model is a bounded asynchronous backend. If the low-confidence arrival rate is $p(\theta)R$ and the Heavy Model service capacity is C_H , stable operation requires $p(\theta)R \leq C_H$. A deployment can bound the slow-path queue and handle excess requests according to its latency and accuracy requirements while retaining Light Model service on the fast path.

When the low-confidence buffer exceeds a threshold or redirection frequency increases significantly, incremental distillation is triggered. The Light Model is fine-tuned on newly collected samples with the Heavy Model as teacher, focusing on updating the gating network and 1D-CNN kernels. Updated parameters are then pushed to edge devices for seamless deployment. This closed-loop design ensures sustained performance in dynamic environments while minimizing manual re-labeling costs.

5 Evaluation

We evaluate Prometheus as a deployable encrypted-traffic analysis system, focusing on classification quality, heavy-light runtime behavior, online adaptation, and end-to-end performance. The evaluation centers on whether the system can sustain real 10 Gbps processing, how it scales on a 100 Gbps-capable multi-GPU platform, and where bottlenecks emerge along the NIC/host/GPU path.

5.1 Experimental Setup

Unless otherwise specified, model training, classification experiments, and ablation studies are conducted in PyTorch on a server with eight NVIDIA V100 GPUs based on the Volta architecture. The end-to-end deployment and scaling testbeds are described separately in Sec. 5.4 and Sec. 5.5. Model architecture and hyperparameter settings are detailed in the supplementary material.

Datasets. We pre-trained our model on MAWI [13] and CIC-IoT-2022 [15] (68 GB raw PCAP, 3M bidirectional flows), and evaluate on CipherSpectrum [58], CSTNET-TLS 1.3 [34], and DataCon2021 [16], covering modern TLS 1.3 variants, real-world web traffic, and encrypted proxy traces. Full dataset statistics and descriptions are provided in the supplementary material. The downstream labels correspond to the operational task represented by each dataset: application/domain

Table 2. Core classification results (macro-F1). Full AC/PR/RC/F1 metrics are provided in the supplementary material.

Method	CST	A-128	A-256	CHA	MIX	DataC	Avg.
ET-BERT	0.865	0.604	0.616	0.559	0.328	0.875	0.641
YaTC	0.899	0.895	0.873	0.800	0.828	0.824	0.853
TrafficFormer	0.852	0.408	0.454	0.404	0.383	0.896	0.566
NetMamba	0.809	0.802	0.757	0.733	0.759	0.060	0.653
Ours	0.939	0.942	0.941	0.936	0.922	0.958	0.940

identification for CipherSpectrum and CSTNET-TLS 1.3, cipher-variant robustness across the AES-128, AES-256, ChaCha20, and mixed subsets, and encrypted-proxy traffic identification for DataCon2021. These are flow-level predictions intended for monitoring and traffic-management decisions; they do not require plaintext inspection.

Data Processing. Raw traffic is converted into bidirectional flows via *SplitCap*. Following the evaluation standard of Zhao et al. [76], we use a per-flow split: all packets belonging to one flow remain in exactly one partition, preventing implicit flow identifiers from leaking across train, validation, and test sets. We mask “shortcut features” (e.g., IPs/ports) and use a 7:1.5:1.5 train/validation/test ratio. To support early inference, flows are truncated to the first 10 packets and 300 bytes per packet.

Baselines & Metrics. We compare our method against representative state-of-the-art paradigms, including ET-BERT [34], TrafficFormer [78] (Transformer-based), YaTC [75] (MAE-based), and NetMamba [55] (SSM-based). We use the official implementations and released checkpoints when available, fine-tune all models on the same downstream partitions, and provide the same first-10-packet/300-byte-per-packet budget. Throughput uses the same replay, batching, and measurement procedure for every model. Classification quality is evaluated using accuracy, macro-averaged precision, recall, and F1-score. System performance is evaluated using sustainable input bandwidth, end-to-end batch latency, Heavy Model call rate, latency breakdown, and multi-GPU scaling.

5.2 Classification Performance Comparison

We evaluate Heavy Model performance against baselines under full fine-tuning on six datasets spanning standard TLS 1.3 traffic, multiple cipher suites, mixed-encryption scenarios, and encrypted proxy-traffic analysis. Tab. 2 reports the macro-F1 scores most relevant to model quality; full metric breakdowns are deferred to the supplementary material.

On standard CSTNET-TLS 1.3, our method reaches 93.9% F1, outperforming YaTC by 4.0 points. Across different cipher suites and mixed-encryption scenarios, most baselines degrade substantially. TrafficFormer drops from 85.2% to 40.8% F1 on TLS-AES-128-GCM, whereas our method maintains over 92% F1 across all variants. This stability suggests that our model captures encryption-invariant patterns rather

Table 3. Per-class Heavy Model error breakdown. We report the lowest-F1 class in each test set and its largest outgoing confusion; the three lowest-F1 classes per dataset, with precision, recall, and support, are detailed in the supplementary material.

Dataset	Hardest class	Recall / F1 / support	Largest outgoing confusion
AES-128-GCM	googletagmanager.com	0.467 / 0.636 / 150	google-analytics.com (80)
AES-256-GCM	googletagmanager.com	0.527 / 0.690 / 150	google-analytics.com (71)
ChaCha20-Poly1305	googletagmanager.com	0.380 / 0.545 / 150	google-analytics.com (93)
Mixed	googletagmanager.com	0.612 / 0.671 / 152	google-analytics.com (59)
CSTNET-TLS 1.3	class 74	0.444 / 0.571 / 18	classes 11 and 65 (3 each)
DataCon2021	netch (class 10)	0.867 / 0.839 / 15	v2ray (2)

than cipher-specific artifacts. On DataCon2021, which contains encrypted proxy traffic with heterogeneous tunnel behaviors, we achieve 95.8% F1 versus TrafficFormer’s 89.6%.

We analyze the most difficult classes in each dataset to identify where classification errors occur and what traffic characteristics contribute to them. Table 3 shows that errors are concentrated in a small number of classes within each dataset. In all four CipherSpectrum variants, google tagmanager .com shows the same error pattern: it is most often confused with google-analytics.com. This persistent, asymmetric confusion across cipher suites suggests overlap in their early-flow side-channel patterns; the supplementary material examines this pair in detail. CSTNET uses anonymous numeric labels, so we report its difficult classes by ID. In DataCon2021, WireGuard and OpenVPN-over-UDP are classified correctly in the held-out test set, while the remaining errors concentrate on Netch and V2Ray (F1 0.839 and 0.867), two multi-protocol proxy clients whose observable behavior depends on the relayed traffic.

Under frozen-encoder evaluation, our method consistently outperforms baselines; notably, it maintains 67.19% accuracy on AES-128 where YaTC and TrafficFormer drop below 12%, demonstrating that protocol-aware inductive biases enable transferable representations.

5.3 Collaborative Inference and Online Adaptation

We next evaluate the runtime collaboration policy. The goal is not only to improve aggregate accuracy, but also to verify that the Heavy Model is invoked selectively and that online distillation reduces sustained backend pressure after distribution shifts.

Overall Performance Gain. To quantify the benefits of the collaborative inference mechanism, we designed a comparative experiment across six datasets. We measured the classification accuracy of the standalone Light Model and compared it against the deployed online dual-model system with a fixed confidence threshold of 0.5, to verify whether the Heavy Model can effectively compensate for the Light Model’s capacity constraints on complex traffic samples.

As illustrated in Fig. 5, the collaborative system exhibits a consistent performance advantage over the standalone Light Model across all datasets. The accuracy improvements range

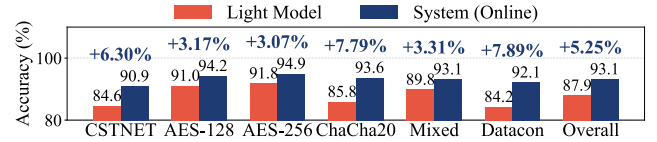


Figure 5. Comparison of classification accuracy between the standalone Light model and the online dual-model system across six datasets (confidence threshold $\theta = 0.5$).

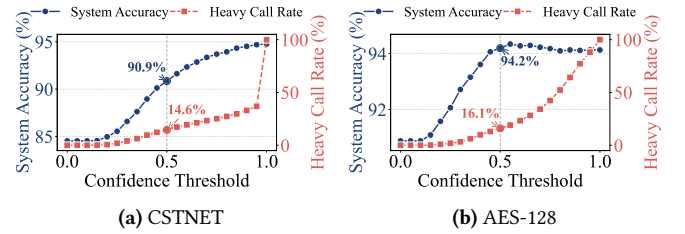


Figure 6. Analysis of the Accuracy-Efficiency Trade-off. The plots demonstrate how the confidence threshold regulates the balance between System Accuracy and Heavy Call Rate.

from 3.07% (on AES-256) to 7.89% (on Datacon). Notably, the system boosts accuracy by over 5% on CSTNET and in the overall average across the six datasets. These results validate that the collaborative architecture effectively leverages the Heavy Model’s superior representation capability to rectify predictions for hard samples, thereby establishing a higher upper bound for system performance.

We further investigated the trade-off between classification precision and computational overhead. The core control knob in our framework is the confidence threshold (θ), which dictates the aggressiveness of offloading traffic to the Heavy Model. We varied θ from 0 to 1 and recorded two metrics: System Accuracy (left y-axis) and Heavy Call Rate (right y-axis, representing the percentage of flows processed by the Heavy Model).

Fig. 6 depicts this sensitivity analysis on the CSTNET and AES-128 datasets. As the confidence threshold increases, the system adopts a more conservative strategy, forwarding more uncertain samples to the Heavy Model, which results in higher accuracy but increased computational cost. Taking the AES-128 dataset (Fig. 6b) as a case study, $\theta = 0.5$ yields 94.2% accuracy with a Heavy Call Rate of only 16.1%. The

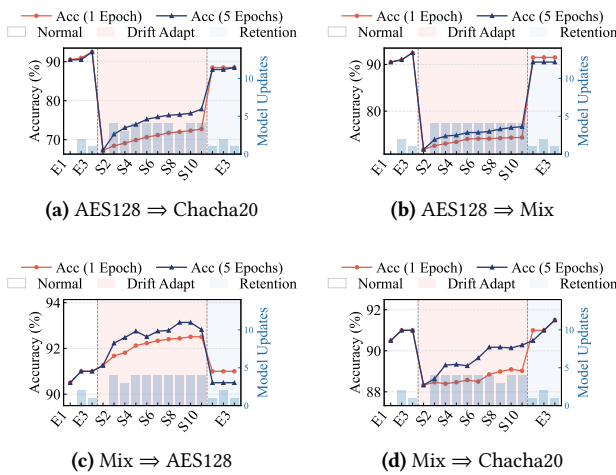


Figure 7. Performance recovery of the Light Model under concept drift. The curves track accuracy evolution across the Normal, Drift Adapt, and Retention phases. Online distillation enables the model to adapt to new encryption patterns, where the 5-epoch strategy (blue) exhibits faster convergence compared to the 1-epoch strategy (red).

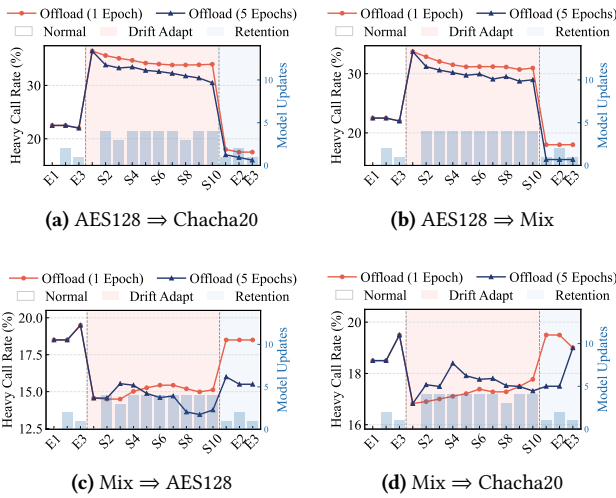


Figure 8. Evolution of system overhead during concept drift adaptation. While the Heavy Model invocation rate typically spikes on novel traffic, it may decrease when transitioning to a less complex distribution.

marginal accuracy dip thereafter likely stems from stochastic instability introduced by frequent online updates on noisy, low-confidence samples. This operating point demonstrates that our framework achieves high-precision classification (exceeding the standalone Light Model) with minimal resource overhead, allowing administrators to flexibly balance performance against constraints.

Adaptation to Evolving Environments. Beyond static performance, we investigate the system’s robustness under evolving network environments. We define concept drift as

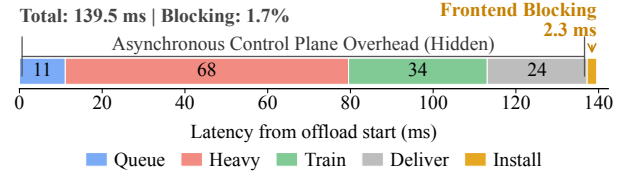


Figure 9. Breakdown of the asynchronous online-update feedback loop in a controlled trace. Heavy inference, student training, and weight delivery run in the control plane; only the final Light Model installation blocks the frontend.

scenarios where the traffic distribution shifts across encryption suites (e.g., from AES-128 to ChaCha20). As illustrated in Fig. 7, our experiment is conducted in three distinct phases: (1) Normal, representing the original distribution; (2) Drift Adapt, where the system encounters and learns from drifted traffic; and (3) Retention, where we re-test the original traffic to ensure no catastrophic forgetting has occurred. For each drift scenario, we partition the data into k segments, performing online learning on the first $k-1$ unlabeled segments; ground-truth labels are held out and used only to compute the reported accuracy curves. We compare two learning intensities: 1 epoch (red line) for rapid adaptation and 5 epochs (blue line) to simulate scenarios with higher data availability.

Performance Recovery is evident in Fig. 7. In the challenging AES-128 ⇒ ChaCha20 transition (Fig. 7a), Light Model accuracy plummets from 90.5% to 67.28% upon drift, but progressively recovers to 72.71% (1 epoch) and 77.83% (5 epochs) by segment S10. This significant improvement demonstrates that the Light Model effectively internalizes the Heavy Model’s protocol-aware knowledge through online distillation, with higher learning intensity further accelerating the adaptation process. Crucially, in the Retention phase, accuracy restores to 88.5%, confirming that the framework successfully accommodates new patterns while maintaining high performance on the original distribution without catastrophic forgetting.

Simultaneously, *System Overhead Reduction* is analyzed in Fig. 8. While the Heavy Call Rate (HCR) typically spikes upon drift, it exhibits a significant drop in scenarios (c) and (d). This is because transitioning from a high-entropy “Mix” environment to a homogeneous single-protocol distribution reduces classification ambiguity, allowing the Light Model to operate with higher inherent confidence. Taking Fig. 8a as an example, the HCR spikes at S2 (36.4%) but progressively declines as the Light Model internalizes new patterns; by S10, the 5-epoch strategy achieves a faster reduction (30.5%) than the 1-epoch baseline (33.9%). Finally, the rate subsides to 16.5% in the Retention phase, confirming sustainable resource efficiency in fluctuating environments.

Non-blocking update path. The online update loop is designed so that expensive work remains off the data path. Fig. 9 reports a breakdown from a controlled online-update trace. The first closed-loop update from difficult-flow offload

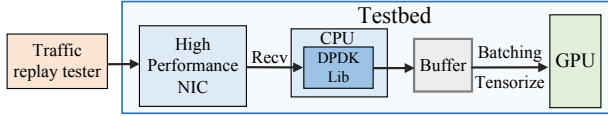


Figure 10. The architecture of real-world testbed.

to frontend weight installation completes in 139.5 ms, but Heavy Model inference, student training, and weight delivery run asynchronously in the control plane. The only frontend-visible step is atomically installing the new Light Model weights, which takes 2.3 ms. This supports the design goal that model evolution can proceed without turning the frontend classifier into a blocking retraining pipeline.

5.4 System Throughput and Latency

Real-world testbed. To evaluate practical deployment performance, we build the 10 Gbps testbed shown in Fig. 10. A TOYO Corporation SYNESIS traffic tester replays dataset PCAPs to a GPU server through an Intel 82599ES 10 GbE NIC. We implement a C-based DPDK receiver that writes incoming traffic into a 4 GB host-memory buffer; Python worker threads then parse flows, construct PyTorch tensors, and submit GPU inference batches. The batch size of traffic-flow tensors is 256. The server contains an NVIDIA RTX 4090 GPU based on the Ada Lovelace architecture, an Intel Core i9-14900K CPU, and 80 GB of DRAM. This setup exercises the full data path, including packet replay, DPDK receive, CPU preprocessing, host-to-GPU transfer, and model inference.

Evaluation metrics. We evaluate the practical performance of different models using two metrics: the end-to-end bandwidth and latency. We define a model’s end-to-end bandwidth as the maximum input traffic rate (Gbps) that it can sustain without causing buffer overflow, while the latency is the end-to-end delay to complete one data batch on the testbed. This reported latency is the pipeline completion time for a batch of 256 already assembled flow tensors. It therefore covers packet reception, CPU parsing, host-to-GPU transfer, and inference after batching, but does not include the time from a flow’s first packet to collecting its input budget or the waiting time to assemble that flow into a batch. Throughput experiments use rate-controlled PCAP replay without preserving the original inter-packet timing; the packet-collection delays in Table 8 are computed separately from the original trace timestamps.

Evaluation setup. Light and Heavy models are co-deployed on the GPU. We set $\theta = 0.5$ in Prometheus, resulting in a 16% Heavy Model call rate. All baselines are evaluated using the same traffic replay, batching, and measurement methodology.

Comparison and analysis. Fig. 11 depicts the practical inference performance of different methods. The complete Prometheus system (Light+Heavy in Fig. 11) sustains 9.8 Gbps end-to-end bandwidth with 2.1 ms traffic-analysis latency, achieving 1.97× the throughput of NetMamba (4.98 Gbps).

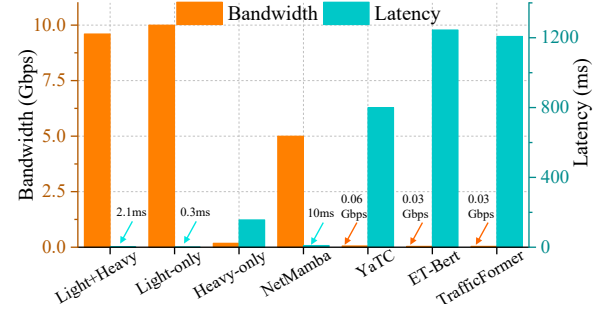


Figure 11. The practical performance in real-world testbed.

The Light-only configuration reaches the 10 Gbps link rate with 0.3 ms latency, corresponding to 333.3× the throughput of ET-BERT and TrafficFormer (0.03 Gbps each). In contrast, the majority of baselines exhibit much lower inference performance than Prometheus: YaTC, ET-BERT, and TrafficFormer deliver less than 100 Mbps bandwidth with latency over 600 ms. With a lightweight state-space architecture, NetMamba reaches 4.98 Gbps bandwidth. However, its inference latency is 4.7× slower than that of Prometheus.

The ten-packet budget is a deliberately fixed operating point that exposes enough bidirectional handshake and early application dynamics while keeping preprocessing and transfer bounded. The supplementary five-packet study shows that the Light Model remains competitive with only five packets. On the four CipherSpectrum cipher subsets, collecting ten rather than five packets increases the median collection time from 13.5–15.2 ms to 41.0–44.5 ms (an additional 27.4–29.2 ms), while Heavy Model macro-F1 increases by 5.2–8.5 percentage points. Thus, ten packets is an explicit accuracy–collection-time operating point, not a claim that its latency is universally identical to five packets; CSTNET and DataCon exhibit larger tails and are analyzed separately in the supplementary material.

These results show that strong offline accuracy does not by itself imply deployability in a high-speed traffic-analysis pipeline. By combining a line-rate Light Model with selective Heavy Model invocation and distillation, Prometheus preserves high classification accuracy while remaining within the throughput and latency envelope of a 10 Gbps deployment.

5.5 Bottleneck and Scaling Analysis

The 10 Gbps testbed establishes deployment feasibility and shows baselines below line rate. The 100 Gbps-capable platform therefore focuses on Prometheus scaling and bottlenecks. This platform runs Ubuntu 22.04, DPDK 22.11, and PyTorch 2.1, and contains four of the same RTX 4090 GPUs as the 10 Gbps testbed, an Intel E810-2CQDA2 100 Gbps NIC, dual Intel Xeon 6530 CPUs, 256 GB DRAM, and a Spirent Landslide traffic tester. We measure the full pipeline from traffic replay to GPU inference, including NIC-to-host transfer, CPU feature processing, host-to-GPU transfer, and

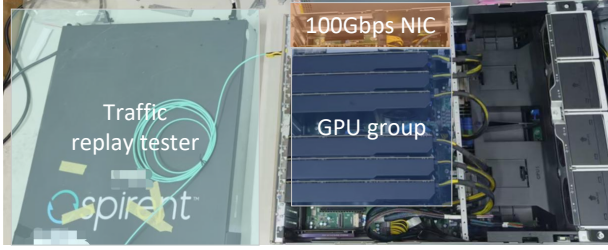


Figure 12. 100 Gbps-capable multi-GPU testbed used for bottleneck and scaling analysis. The setup includes a 100 Gbps traffic generator, a 100 GbE NIC, host CPU preprocessing, PCIe transfer, and four GPUs, allowing us to measure the full path from packet replay to inference.

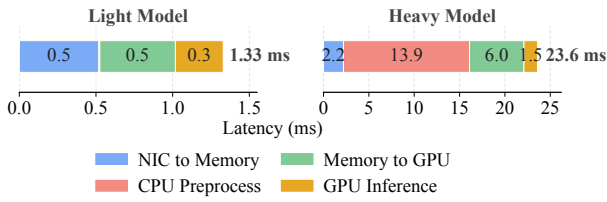


Figure 13. End-to-end latency breakdown on the 100 Gbps-capable testbed. The Light Model reduces both preprocessing and transfer cost; for the Heavy Model, CPU feature processing and host-to-GPU transfer dominate total latency.

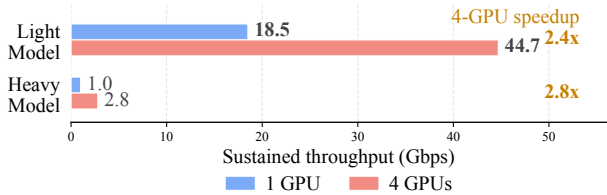


Figure 14. Sustained throughput on the 100 Gbps-capable multi-GPU testbed. The distilled Light Model scales from 18.5 Gbps on one GPU to 44.7 Gbps on four GPUs, while the original Heavy Model remains below 3 Gbps even with four GPUs.

model execution. Prometheus uses standard PyTorch operators without GPU-specific kernels and can run directly on other CUDA-capable GPUs; the heavy-light architecture remains unchanged across GPU platforms.

Latency breakdown. Fig. 13 decomposes end-to-end latency for the distilled Light Model and the original Heavy Model. For the Light Model, the full pipeline takes 1330 μ s: NIC-to-memory transfer takes 518 μ s, CPU preprocessing takes 7 μ s, host-to-GPU transfer takes 493 μ s, and GPU inference takes 312 μ s. Thus, inference itself is no longer the dominant cost; PCIe and host-memory movement account for most of the remaining latency. For the Heavy Model, total latency increases to 23.6 ms, with 13.9 ms spent in CPU preprocessing and 6.0 ms in host-to-GPU transfer. These two stages account for 84.3% of the Heavy Model pipeline, showing that the large feature representation required by the

original model stresses CPU extraction and data movement before GPU compute becomes the limiting factor.

Implication: data movement dominates at 100 Gbps scale. The 100 Gbps-capable testbed shows that after model compression, the limiting path is no longer only GPU inference. Packets still traverse the NIC, host DRAM, CPU preprocessing, PCIe, GPU memory, and GPU kernels; for the optimized Light Model, NIC-to-memory and host-to-GPU transfers already exceed GPU inference time. For the Heavy Model, the larger representation further amplifies CPU feature extraction and host-to-GPU transfer, explaining why adding GPUs alone does not turn it into a viable uniform fast path. This points to a systems requirement beyond model design: future high-speed traffic-analysis appliances need lighter feature representations, feature-extraction offload, or deployment-specific zero-copy paths. GPUDirect RDMA-like mechanisms are a natural direction, but ordinary gateway or Internet traffic does not always match RDMA-oriented deployment assumptions, so the data path must be co-designed with the observation point.

Multi-GPU scaling. Fig. 14 compares single- and four-GPU throughput. The distilled Light Model reaches 18.5 Gbps on one GPU and 44.7 Gbps on four GPUs. In contrast, the Heavy Model reaches only 1.0 Gbps on one GPU and 2.8 Gbps on four GPUs. This gap explains why the Heavy Model cannot serve as the uniform fast path in a high-speed appliance, even when additional GPUs are available. The Light Model’s scaling is also sublinear, which is expected because data movement and CPU-side flow preparation remain shared bottlenecks. These results support the heavy-light architecture: the Light Model provides the scalable data-plane path, while the Heavy Model is reserved for low-confidence flows where its semantic capacity is worth the backend cost.

5.6 Understanding the Design Choices

The preceding experiments show that Prometheus is accurate, adaptive, and deployable. We finally examine why the model design works: whether it learns protocol-relevant structure rather than dataset artifacts, and whether the main architectural components are necessary. We analyze semantic behavior in the main text through dynamic gating and service characterization; detailed head-coefficient visualizations and probing-task results are provided in the supplementary material.

Service-Granularity Behavior Analysis. We further aggregate Layer 4 gating weights by service type on the AES-128 dataset. Fig. 15 reveals four recurrent *Head-Preference Topologies*: periodic services emphasize Temporal and Size heads, static retrievals favor Size, complex infrastructures activate Content/TCP cues, and sparse micro-interactions form low-activation fingerprints. These patterns connect the learned routing behavior to concrete communication primitives; detailed per-service interpretations are provided in the supplementary material.

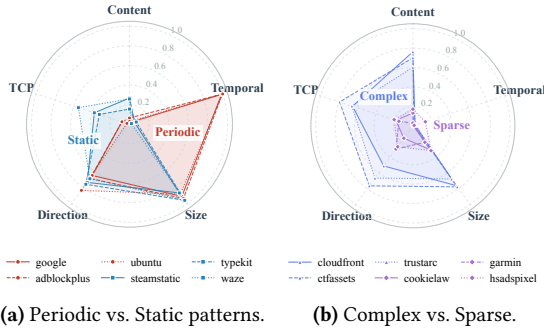


Figure 15. Adaptive head preference topologies across service categories. (a) **Periodic Automation** emphasizes temporal-size regularity, while **Static Retrieval** prioritizes size invariance. (b) **Complex Infrastructure** leverages residual handshake metadata, whereas **Sparse Micro-interactions** exhibit distinctive low-activation footprints.

Component Ablation. We conduct an ablation study to assess each key component in Prometheus (Tab. 4); Head 1–4 denote Temporal, Size, Direction, and TCP, respectively.

Table 4. Ablation study (F1-score).

Method	CST	A-128	A-256	CHA	MIX	DataC	Avg.
Ours-Base	0.939	0.942	0.941	0.936	0.922	0.958	0.940
w/o PT	0.853	0.931	0.596	0.913	0.897	0.245	0.739
w/o 4 Heads	0.915	0.931	0.925	0.919	0.884	0.958	0.922
w/o Head 1	0.928	0.929	0.922	0.919	0.882	0.953	0.922
w/o Head 2	0.918	0.931	0.941	0.935	0.923	0.941	0.931
w/o Head 3	0.927	0.936	0.943	0.933	0.913	0.952	0.934
w/o Head 4	0.929	0.945	0.937	0.930	0.910	0.958	0.935
w/o Gate	0.922	0.928	0.939	0.915	0.889	0.961	0.926

Pre-training is essential: removing it reduces average F1 from 0.940 to 0.739. The largest drop occurs on DataCon (from 0.958 to 0.245), the smallest downstream dataset with approximately 1K flows, where training the Heavy Model from scratch provides insufficient supervision.

Protocol-guided attention heads provide complementary benefits. Removing all four heads reduces average F1 by 1.8%. Individual ablation yields dataset-dependent drops (average 0.5%–1.8% per head), confirming the value of injecting structured priors: Temporal Head (capturing packet timing proximity), Size Head (packet length similarity), Direction Head (directional consistency), and TCP Head (request-response and interaction semantics). The modest drop from removing a single head should be interpreted in the context of head-aligned pre-training: the remaining heads and multi-task objectives have already encoded correlated side-channel structures, so they can partially compensate for one missing protocol cue. To further separate architecture from generic pre-training, we also evaluate a vanilla pre-training control that uses the same data and training budget but removes the four protocol-aware heads and their aligned tasks; on AES-128, F1 drops from 94.2% to 85.0%. This shows that

the benefit comes from the joint protocol-aware representation rather than from any single isolated head or from pre-training alone.

Adaptive gating further improves robustness in mixed or drifting traffic: disabling it lowers average F1 by 1.4%, with a more pronounced drop on TLS-MIX (3.3%). This supports dynamic re-weighting of protocol cues under non-stationary conditions.

6 Discussion and Limitations

Operating limits. Prometheus treats the Heavy Model as a scarce backend resource rather than an always-available oracle. When the Light Model error rate or low-confidence rate rises, operators can increase Heavy Model coverage, adjust the confidence threshold, or apply admission control to protect the receive path. If low-confidence traffic exceeds the configured Heavy Model budget for a sustained period, the system must trade accuracy, latency, and loss by returning Light Model predictions, batching slow-path requests, dropping excess slow-path work, or provisioning more GPUs. Our prototype exposes Heavy Call Rate and the confidence threshold as control knobs, but does not yet implement a full admission-control policy.

Evasion and the arms race. Prometheus relies only on observable side channels—packet sizes, timing, directions, and transport interactions—and does not inspect plaintext payloads. An adversary who wishes to avoid identification must therefore coordinate manipulation across several of these dimensions (e.g., padding, timing and length jitter, direction flipping) while preserving transmission semantics and protocol constraints, which raises the cost of evasion. The supplementary material shows that Prometheus degrades more gracefully than prior methods under byte masking, length jitter, direction flipping, and timing jitter. Highly coordinated countermeasures such as cover traffic and onion routing constitute a stronger active-adversary setting that lies outside of our current evaluation. It is left to future work.

Beyond standard metrics, we also evaluate Prometheus’s robustness against dataset-specific artifacts and common traffic perturbations, and conduct a fairness study under restricted input budgets. These results are deferred to the supplementary material.

7 Related Work

Encrypted-traffic classifiers. As encryption protocols evolved, traffic classification shifted from port-based and DPI approaches [3, 18, 48] to learned classifiers over observable side channels. Existing methods include supervised byte models using CNNs [8, 19, 37, 56, 59], RNNs [17, 31, 57, 60], Transformers [43, 74, 77], and GNNs [14, 22, 62, 70, 71]; statistical feature models [4, 7, 20, 36, 69] supported by realistic NetFlow generation [54]; and sequence models over packet lengths [6, 12, 42, 51, 52, 63], message types [30, 47],

Table 5. Representative approaches to traffic classification. “No” means that no online model-update loop is reported. The supplementary material provides the full comparison, including protocol priors.

Approach	Task / scope	Model / representation	Execution	Online adaptation
TrafficFormer [78] YaTC [75] NetMamba [55]	Encrypted traffic classification	Transformer, masked autoencoder, or state-space model	CPU/GPU inference	No
Akem et al. [1]	Encrypted traffic classification	Random forest; packet size and arrival time	P4/Tofino switch	No
Brain-on-Switch [67]	Line-rate traffic analysis	Compact NN; packet length and delay	Switch-first with off-switch escalation	No
Prometheus	Encrypted flow classification	Protocol-aware Heavy Model; distilled Light Model	DPDK CPU + GPU backend	Asynchronous distillation

and timing [2, 5, 45, 66] for anonymous [27, 40, 49] and encrypted traffic [9, 30, 35, 39, 61, 64, 65]. Pre-training further improves transferability with token or image-like masked modeling [26, 28, 34, 75, 78], efficient sequence architectures [55], and multi-modal objectives over bytes, timing, packet lengths, and metadata [24, 32, 33, 41, 68]. These classifiers improve representation quality, but they are usually evaluated as offline models or uniform inference pipelines. Prometheus instead treats representation depth as a scheduled backend resource: a compact Light Model serves the data path, while a protocol-aware Heavy Model is invoked selectively and distilled online. tFusion [21] is also relevant as a cross-modality representation approach for few-shot malicious-traffic detection; unlike our flow-classification setting, it does not study a heavy–light serving split or online distillation. Table 5 summarizes representative modeling and deployment approaches. The full comparison in the supplementary material also covers MM4Flow, tFusion, Taurus, and Quark, including their protocol priors.

Systems for Line-Rate Traffic Analytics. Systems work accelerates traffic analytics with edge accelerators and programmable data planes. FENXI decouples forwarding from flow-level deep-learning analytics and uses asynchronous communication plus dynamic batching for 100 Gbps edge processing [23]; other systems push per-packet ML, switch-based analysis, CNN execution, and monitoring into programmable data planes [50, 67, 72, 79]. These systems demonstrate the importance of line-rate execution, but they often favor compact models constrained by data-plane resources. Prometheus takes a complementary approach: it keeps the fast path compact for throughput, while preserving a backend Heavy Model and online distillation loop for representation depth and adaptation.

Data-plane-only comparison. Encrypted-traffic classifiers such as Akem et al. [1] encode compact packet-size and arrival-time features in programmable switches, enabling line-rate inference with sub-microsecond delay but limiting model capacity to switch memory and arithmetic budgets. Brain-on-Switch [67] and Quark [72] demonstrate switch-native neural inference; Brain-on-Switch is a switch-first

hybrid that can escalate selected traffic to an off-switch Transformer. Prometheus is complementary: it reserves a compact Light Model for the fast path, invokes a richer GPU-backed Heavy Model selectively, and distills its outputs asynchronously for online adaptation. The two system families and their relationship to Prometheus are summarized in Table 5.

Feature-extraction substrates. Recent systems such as SuperFE and Phantom optimize the measurement substrate by offloading flexible feature computation or virtualizing switch resources [10, 73]. These systems are complementary to Prometheus: they can reduce CPU-side feature extraction and measurement overhead, while Prometheus studies encrypted-traffic classification as a complete heavy-light inference pipeline, including confidence routing, online distillation, and NIC/host/GPU data movement on 10–100 Gbps testbeds.

8 Conclusion

We present Prometheus, a protocol-aware heavy-light collaborative system for real-time encrypted traffic classification. By decoupling semantic inspection (Heavy Model) from high-throughput inference (Light Model) and connecting them with confidence-based routing and online distillation, Prometheus resolves the practical tension between representation robustness and throughput under load spikes and distribution shifts. Extensive experiments across diverse encrypted traffic datasets show that our protocol-guided multimodal attention and head-aligned pre-training improve both accuracy and transferability, while the Light Model can continuously evolve under concept drift with limited heavy-model offloading.

Acknowledgments

We sincerely thank the anonymous Shepherd and reviewers for their constructive comments. This work was supported in part by the Major Key Project of Pengcheng Laboratory (PCL) under grant No. PCL2026A01, in part by the Basic and Frontier Research Project of PCL under grant No. 2025QYB004, and in part by the National Natural Science Foundation of China (NSFC) under grant No. 62402012.

References

- [1] Aristide Tanyi-Jong Akem, Guillaume Fraysse, and Marco Fiore. 2024. Encrypted Traffic Classification at Line Rate in Programmable Switches with Machine Learning. In *2024 IEEE Network Operations and Management Symposium (NOMS)*. IEEE, 1–9. doi:10.1109/NOMS59830.2024.10575394
- [2] Khaled Al-Naami, Swarup Chandra, Ahmad Mustafa, Latifur Khan, Zhiqiang Lin, Kevin Hamlen, and Bhavani Thuraisingham. 2016. Adaptive encrypted traffic fingerprinting with bi-directional dependence. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC '16)*. ACM, 177–188.
- [3] Shane Alcock and Richard Nelson. 2012. *Libprotoident: traffic classification using lightweight packet inspection*. Technical Report. Technical report, University of Waikato.
- [4] Blake Anderson and David McGrew. 2017. Machine learning for encrypted malware traffic classification: accounting for noisy labels and non-stationarity. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '17)*. ACM, 1723–1732.
- [5] Rushi Jayeshkumar Babaria, Minzhao Lyu, Gustavo Batista, and Vijay Sivaraman. 2025. FastFlow: Early Yet Robust Network Flow Classification using the Minimal Number of Time-Series Packets. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 2 (2025), 1–27.
- [6] Diogo Barradas, Nuno Santos, Luís Rodrigues, Salvatore Signorello, Fernando MV Ramos, and André Madeira. 2021. FlowLens: Enabling Efficient Flow Classification for ML-based Network Security Applications. In *Network and Distributed Systems Security (NDSS) Symposium*.
- [7] Riccardo Bortolameotti, Thijs van Ede, Marco Caselli, Maarten H Everts, Pieter Hartel, Rick Hofstede, Willem Jonker, and Andreas Peter. 2017. Decanter: Detection of anomalous outbound http traffic by passive application fingerprinting. In *Proceedings of the 33rd Annual Computer Security Applications Conference (ACSAC '17)*. ACM, 373–386.
- [8] Zhiyong Bu, Bin Zhou, Pengyu Cheng, Kecheng Zhang, and Zhen-Hua Ling. 2020. Encrypted network traffic classification using deep and parallel network-in-network models. *IEEE Access* 8 (2020), 132950–132959.
- [9] Bogdan Cebere and Christian Rossow. 2024. Understanding Web Fingerprinting with a Protocol-Centric Approach. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '24)*. ACM, 17–34.
- [10] Xiang Chen, Hongyan Liu, Zhengyan Zhou, Xi Sun, Wenbin Zhang, Hongyang Du, Dong Zhang, Xuan Liu, Haifeng Zhou, Dusit Niyato, Qun Huang, Chunming Wu, and Kui Ren. 2025. Phantom: Virtualizing Switch Register Resources for Accurate Sketch-based Network Measurement. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 1383–1398. doi:10.1145/3689031.3696077
- [11] Xu-Yang Chen, Lu Han, De-Chuan Zhan, and Han-Jia Ye. 2025. MIETT: Multi-Instance Encrypted Traffic Transformer for Encrypted Traffic Classification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 15922–15929.
- [12] Yige Chen and Yipeng Wang. 2024. MPAF: Encrypted traffic classification with multi-phase attribute fingerprint. *IEEE Transactions on Information Forensics and Security* 19 (2024), 7091–7105.
- [13] Kenjiro Cho, Koushirou Mitsuya, and Akira Kato. 2000. Traffic data repository at the WIDE project. In *USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association.
- [14] Susu Cui, Xueying Han, Dongqi Han, Zhiliang Wang, Weihang Wang, Bo Jiang, Baoxu Liu, and Zhigang Lu. 2025. FG-SAT: Efficient Flow Graph for Encrypted Traffic Classification under Environment Shifts. *IEEE Transactions on Information Forensics and Security* (2025).
- [15] Sajjad Dadkhah, Hassan Mahdikhani, Priscilla Kyei Danso, Alireza Zohourian, Kevin Anh Truong, and Ali A Ghorbani. 2022. Towards the development of a realistic multidimensional IoT profiling dataset. In *2022 19th Annual International Conference on Privacy, Security & Trust (PST)*. IEEE, 1–11.
- [16] DataCon Community. 2021. DataCon2021: Encrypted Proxy Traffic Dataset. <https://datacon.qianxin.com/opendata/openpage?resourceId=10>. Accessed: 2021.
- [17] Xianwen Deng, Yijun Wang, and Zhi Xue. 2024. An-net: An anti-noise network for anonymous traffic classification. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*. ACM, 4417–4428.
- [18] Luca Deri, Maurizio Martinelli, Tomasz Bujlow, and Alfredo Cardigliano. 2014. ndpi: Open-source high-speed deep packet inspection. In *2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*. IEEE, 617–622.
- [19] Kevin Fauvel, Fuxing Chen, and Dario Rossi. 2023. A lightweight, efficient and explainable-by-design convolutional neural network for internet traffic classification. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. ACM, 4013–4023.
- [20] Alessandro Finamore, Marco Mellia, Michela Meo, and Dario Rossi. 2010. Kiss: Stochastic packet inspection classifier for udp traffic. *IEEE/ACM Transactions on Networking* 18, 5 (2010), 1505–1515.
- [21] Chuanpu Fu, Qi Li, Elisa Bertino, and Ke Xu. 2025. Training with Only 1.0% Samples: Malicious Traffic Detection via Cross-Modality Feature Fusion. In *Proceedings of the 32nd ACM SIGSAC Conference on Computer and Communications Security*. ACM, 3930–3944. doi:10.1145/3719027.3765143
- [22] Chuanpu Fu, Qi Li, and Ke Xu. 2023. Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis. *arXiv preprint arXiv:2301.13686* (2023).
- [23] Massimo Gallo, Alessandro Finamore, Gwendal Simon, and Dario Rossi. 2021. FENXI: Deep-learning Traffic Analytics at the Edge. In *Proceedings of the Sixth ACM/IEEE Symposium on Edge Computing (SEC '21)*. ACM, 202–216. doi:10.1145/3453142.3491273
- [24] Satyandra Guthula, Roman Beltiukov, Navya Battula, Wenbo Guo, and Arpit Gupta. 2023. netFound: Foundation model for network security. *arXiv preprint arXiv:2310.17025* (2023).
- [25] Yi Han, Matthew Chan, Zahra Aref, Nils Ole Tippenhauer, and Saman Zonouz. 2022. Hiding in plain sight? on the efficacy of power side {Channel-Based} control flow monitoring. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 661–678.
- [26] Zijun Hang, Yuliang Lu, Yongjie Wang, and Yi Xie. 2023. Flow-mae: Leveraging masked autoencoder for accurate, efficient and robust malicious traffic classification. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '23)*. ACM, 297–314.
- [27] Jamie Hayes and George Danezis. 2016. k-fingerprinting: A robust scalable website fingerprinting technique. In *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, 1187–1203.
- [28] Hong Ye He, Zhi Guo Yang, and Xiang Ning Chen. 2020. PERT: Payload encoding representation from transformer for encrypted traffic classification. In *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, 1–8.
- [29] Jana Iyengar, Martin Thomson, et al. 2021. QUIC: A UDP-based multiplexed and secure transport. In *RFC 9000*.
- [30] Maciej Korczyński and Andrzej Duda. 2014. Markov chain fingerprinting to classify encrypted traffic. In *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 781–789.
- [31] Rui Li, Xi Xiao, Shiguang Ni, Haitao Zheng, and Shutao Xia. 2018. Byte segment neural network for network traffic classification. In *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.

- [32] Xiang Li, Juncheng Guo, Qige Song, Jiang Xie, Yafei Sang, Shuyuan Zhao, and Yongzheng Zhang. 2023. Listen to minority: Encrypted traffic classification for class imbalance with contrastive pre-training. In *2023 20th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. IEEE, 447–455.
- [33] Peng Lin, Kejiang Ye, Yishen Hu, Yanying Lin, and Cheng-Zhong Xu. 2022. A novel multimodal deep learning framework for encrypted traffic classification. *IEEE/ACM Transactions on Networking* 31, 3 (2022), 1369–1384.
- [34] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. 2022. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*. ACM, 633–642.
- [35] Chang Liu, Zigang Cao, Gang Xiong, Gaopeng Gou, Siu-Ming Yiu, and Longtao He. 2018. Mampf: Encrypted traffic classification based on multi-attribute markov probability fingerprints. In *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.
- [36] Huisheng Liu, Zhenxing Wang, and Yu Wang. 2012. Semi-supervised encrypted traffic classification using composite features set. *Deakin University* (2012).
- [37] Mohammad Lotfollahi, Mahdi Jafari Siavoshani, Ramin Shirali Hossein Zade, and Mohammadsadegh Saberian. 2020. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing* 24, 3 (2020), 1999–2012.
- [38] Andrew W Moore and Denis Zuev. 2005. Internet traffic classification using bayesian analysis techniques. In *Proceedings of the 2005 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '05)*. ACM, 50–60.
- [39] Wubin Pan, Guang Cheng, and Yongning Tang. 2017. Wenc: Https encrypted traffic classification using weighted ensemble learning and markov chain. In *2017 IEEE Trustcom/BigDataSE/ICSS*. IEEE, 50–57.
- [40] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinnen, Martin Henze, and Klaus Wehrle. 2016. Website Fingerprinting at Internet Scale.. In *Network and Distributed Systems Security (NDSS) Symposium*, Vol. 1. 23477.
- [41] Lingfeng Peng, Xiaohui Xie, Sijiang Huang, Ziyi Wang, and Yong Cui. 2024. PtU: Pre-Trained Model for Network Traffic Understanding. In *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*. IEEE, 1–12.
- [42] Julien Piet, Dubem Nwoji, and Vern Paxson. 2023. Ggfast: Automating generation of flexible network traffic classifiers. In *Proceedings of the ACM SIGCOMM 2023 Conference (SIGCOMM '23)*. ACM, 850–866.
- [43] Xing Qiu, Guang Cheng, Weizhou Zhu, Dandan Niu, and Nan Fu. 2025. Dual-Channel Interactive Graph Transformer for Traffic Classification with Message-Aware Flow Representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 685–693.
- [44] Eric Rescorla. 2018. *The transport layer security (TLS) protocol version 1.3*. Technical Report.
- [45] Tal Shapira and Yuval Shavitt. 2019. Flowpic: Encrypted internet traffic classification is as easy as image recognition. In *IEEE INFOCOM 2019-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 680–687.
- [46] Adit Sharma and Arash Habibi Lashkari. 2025. A survey on encrypted network traffic: A comprehensive survey of identification/classification techniques, challenges, and future directions. *Computer Networks* 257 (2025), 110984.
- [47] Meng Shen, Mingwei Wei, Liehuang Zhu, Mingzhong Wang, and Fuliang Li. 2016. Certificate-aware encrypted traffic classification using second-order markov chain. In *2016 IEEE/ACM 24th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.
- [48] Justine Sherry, Chang Lan, Raluca Ada Popa, and Sylvia Ratnasamy. 2015. Blindbox: Deep packet inspection over encrypted traffic. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*. ACM, 213–226.
- [49] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. 2018. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*. ACM, 1928–1943.
- [50] Tushar Swamy, Alexander Rucker, Muhammad Shahbaz, Ishan Gaur, and Kunle Olukotun. 2022. Taurus: a data plane architecture for per-packet ML. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, 1099–1114.
- [51] Vincent F Taylor, Riccardo Spolaor, Mauro Conti, and Ivan Martinovic. 2016. Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 439–454.
- [52] Rahmadi Trimnanda, Janus Varmarken, Athina Markopoulou, and Brian Demsky. 2020. Packet-level signatures for smart home devices. In *Network and Distributed Systems Security (NDSS) Symposium*, Vol. 2020.
- [53] Thijs Van Ede, Riccardo Bortolameotti, Andrea Continella, Jingjing Ren, Daniel J Dubois, Martina Lindorfer, David Choffnes, Maarten Van Steen, and Andreas Peter. 2020. Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In *Network and Distributed Systems Security (NDSS) Symposium*, Vol. 27.
- [54] Miel Verkerken, Laurens D'hooge, Bruno Volckaert, Filip De Turck, and Giovanni Apruzzese. 2025. ConCap: Practical Network Traffic Generation for Flow-based Intrusion Detection Systems. *arXiv preprint arXiv:2509.16038* (2025).
- [55] Tongze Wang, Xiaohui Xie, Wenduo Wang, Chuyi Wang, Youjian Zhao, and Yong Cui. 2024. Netmamba: Efficient network traffic classification via pre-training unidirectional mamba. In *2024 IEEE 32nd International Conference on Network Protocols (ICNP)*. IEEE, 1–11.
- [56] Wei Wang, Ming Zhu, Jinlin Wang, Xuwen Zeng, and Zhongzhen Yang. 2017. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In *2017 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE, 43–48.
- [57] Xiaorong Wang, Wenting Wei, Xiaoshan Yu, Danyang Zheng, Neeraj Kuma, and Lei Liu. 2024. Ensemble Learning-based Traffic Classification with Small-Scale Datasets for Wireless Networks. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 1–6.
- [58] Nimesha Wickramasinghe, Arash Shaghaghi, Gene Tsudik, and Sanjay Jha. 2025. SoK: Decoding the Enigma of Encrypted Network Traffic Classifiers. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1825–1843.
- [59] Xi Xiao, Shuo Wang, Guangwu Hu, Qing Li, Kelong Mao, Xiapu Luo, Bin Zhang, and Shutao Xia. 2025. RBLJAN: Robust Byte-Label Joint Attention Network for Network Traffic Classification. *IEEE Transactions on Dependable and Secure Computing* 22, 3 (2025), 2161–2178.
- [60] Xi Xiao, Wentao Xiao, Rui Li, Xiapu Luo, Haitao Zheng, and Shutao Xia. 2021. EBSNN: Extended byte segment neural network for network traffic classification. *IEEE Transactions on Dependable and Secure Computing* 19, 5 (2021), 3521–3538.
- [61] Renjie Xie, Yixiao Wang, Jiahao Cao, Enhuan Dong, Mingwei Xu, Kun Sun, Qi Li, Licheng Shen, and Menghao Zhang. 2023. Rosetta: Enabling robust tls encrypted traffic classification in diverse network environments with tcp-aware traffic augmentation. In *Proceedings of the ACM Turing Award Celebration Conference-China 2023 (TURC '23)*. ACM, 131–132.
- [62] Hongbo Xu, Chengxiang Si, Shuhao Li, Zhenyu Cheng, Chenxu Wang, Jiang Xie, Peishuai Sun, and Qingyun Liu. 2025. FlowMiner: A Powerful Model Based on Flow Correlation Mining for Encrypted Traffic Classification. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [63] Shi-Jie Xu, Guang-Gang Geng, Xiao-Bo Jin, Dong-Jie Liu, and Jian Weng. 2022. Seeing traffic paths: Encrypted traffic classification with path signature features. *IEEE Transactions on Information Forensics*

- and Security 17 (2022), 2166–2181.
- [64] Diwen Xue, Michalis Kallitsis, Amir Houmansadr, and Roya Ensafi. 2024. Fingerprinting Obfuscated Proxy Traffic with Encapsulated {TLS} Handshakes. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2689–2706.
 - [65] Diwen Xue, Reethika Ramesh, Arham Jain, Michalis Kallitsis, J. Alex Halderman, Jedidiah R. Crandall, and Roya Ensafi. 2022. OpenVPN is Open to VPN Fingerprinting. In *31st USENIX Security Symposium (USENIX Security 22)*, Kevin R. B. Butler and Kurt Thomas (Eds.). USENIX Association, 483–500.
 - [66] Diwen Xue, Robert Stanley, Piyush Kumar, and Roya Ensafi. 2025. The Discriminative Power of Cross-layer RTTs in Fingerprinting Proxy Traffic. In *Network and Distributed Systems Security (NDSS) Symposium*.
 - [67] Jinzhu Yan, Haotian Xu, Zhuotao Liu, Qi Li, Ke Xu, Mingwei Xu, and Jianping Wu. 2024. {Brain-on-Switch}: Towards advanced intelligent network data plane via {NN-Driven} traffic analysis at {Line-Speed}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, 419–440.
 - [68] Luming Yang, Lin Liu, Junjie Huang, Zhuotao Liu, Shiyu Liang, Shaojing Fu, and Yongjun Wang. 2025. MM4flow: A Pre-trained Multi-modal Model for Versatile Network Traffic Analysis. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*. ACM, 1664–1678.
 - [69] Faiz Zaki, Firdaus Afifi, Shukor Abd Razak, Abdullah Gani, and Nor Badrul Anuar. 2022. GRAIN: Granular multi-label encrypted traffic classification using classifier chain. *Computer Networks* 213 (2022), 109084.
 - [70] Haozhen Zhang, Le Yu, Xi Xiao, Qing Li, Francesco Mercaldo, Xiapu Luo, and Qixu Liu. 2023. Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. In *Proceedings of the ACM Web Conference 2023 (WWW '23)*. ACM, 2066–2075.
 - [71] Haozhen Zhang, Haodong Yue, Xi Xiao, Le Yu, Qing Li, Zhen Ling, and Ye Zhang. 2025. Revolutionizing Encrypted Traffic Classification with MH-Net: A Multi-View Heterogeneous Graph Model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 1048–1056.
 - [72] Mai Zhang, Lin Cui, Xiaoquan Zhang, Fung Po Tso, Zhang Zhen, Yuhui Deng, and Zhetao Li. 2025. Quark: Implementing Convolutional Neural Networks Entirely on Programmable Data Plane. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.
 - [73] Menghao Zhang, Guanyu Li, Cheng Guo, Renyu Yang, Shicheng Wang, Han Bao, Xiao Li, Mingwei Xu, Tianyu Wo, and Chunming Hu. 2025. SuperFE: A Scalable and Flexible Feature Extractor for ML-based Traffic Analysis Applications. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 818–834. doi:10.1145/3689031.3696081
 - [74] Ruijie Zhao, Xianwen Deng, Zhicong Yan, Jun Ma, Zhi Xue, and Yijun Wang. 2022. Mt-flowformer: A semi-supervised flow transformer for encrypted traffic classification. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*. ACM, 2576–2584.
 - [75] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Yanhao Wang, Yijun Wang, Guan Gui, and Zhi Xue. 2023. Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5420–5427.
 - [76] Yuqi Zhao, Giovanni Dettori, Matteo Boffa, Luca Vassio, and Marco Mellia. 2025. The sweet danger of sugar: Debunking representation learning for encrypted traffic classification. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. ACM, 296–310.
 - [77] Ziming Zhao, Zhuoxue Song, Xiaofei Xie, Zhaoxuan Li, Jiongchi Yu, Fan Zhang, and Tingting Li. 2025. Towards Context-Aware Traffic Classification via Time-Wavelet Fusion Network. In *Proceedings of*

the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1 (KDD '25). ACM, 2089–2100.

- [78] Guangmeng Zhou, Xiongwen Guo, Zhuotao Liu, Tong Li, Qi Li, and Ke Xu. 2025. Trafficformer: an efficient pre-trained model for traffic data. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1844–1860.
- [79] Huancheng Zhou and Guofei Gu. 2024. Cerberus: Enabling efficient and effective in-network monitoring on programmable switches. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4424–4439.

Appendices

A Mathematical Definitions of Multi-modal Loss

Packet Type Classification ($\mathcal{L}_{type}$) supervises the model to distinguish between different functional roles of packets within an encrypted tunnel. We simplify this into a binary classification task to identify data-carrying packets (payload > 0) versus control packets (e.g., pure ACKs). The loss is defined as:

$$\mathcal{L}_{type} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)], \quad (17)$$

where $y_i \in \{0, 1\}$ is the ground-truth type and $\hat{y}_i$ is the predicted probability from the content-specialized attention head.

Temporal Distribution Prediction ($\mathcal{L}_{iat}$) encourages the temporal head to capture the burstiness and timing patterns of the traffic. We discretize the Inter-Arrival Times (IAT) into $K = 10$ logarithmic bins, treating it as a multi-class classification problem:

$$\mathcal{L}_{iat} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k}), \quad (18)$$

where $y_{i,k}$ is the one-hot encoded bin index for the i -th packet's IAT. This formulation provides a robust supervision signal that is less sensitive to jitter and extreme outliers than direct regression.

Log-scaled Size Regression ($\mathcal{L}_{size}$) enforces the size-specialized head to reconstruct the packet length metadata. To handle the high dynamic range of packet sizes up to the MTU, we apply a logarithmic transformation to the targets and utilize Mean Squared Error (MSE):

$$\mathcal{L}_{size} = \frac{1}{N} \sum_{i=1}^N \|\hat{s}_i - \log_{10}(s_i + 1)\|^2, \quad (19)$$

where s_i is the original packet size and $\hat{s}_i$ is the regressed value, ensuring the model maintains sensitivity across both small control segments and large data frames.

Next-hop Direction Prediction ($\mathcal{L}_{dir}$) focuses on the sequential dependencies and transmission logic of the protocol. For a given packet representation at index i , the model predicts the direction of the subsequent packet $i + 1$. The

cross-entropy loss is defined as:

$$\mathcal{L}_{dir} = -\frac{1}{N-1} \sum_{i=1}^{N-1} \sum_{c \in C} y_{i+1,c} \log(\hat{y}_{i+1,c}), \quad (20)$$

where $C = \{\text{OUT}, \text{IN}, \text{PADDING}\}$ represents the direction labels, forcing the directional head to learn transition patterns invariant to encryption.

Request–Response Adjacency ($\mathcal{L}_{pair}$) identifies semantic dependencies between adjacent packets, specifically detecting if they constitute a round-trip interaction. By concatenating the representations of packets $(i, i+1)$, the semantics head predicts the adjacency state:

$$\mathcal{L}_{pair} = -\frac{1}{N-1} \sum_{i=1}^{N-1} \sum_{c \in \{0,1\}} y_{i,c} \log(\hat{y}_{i,c}), \quad (21)$$

where $y_{i,pair} = 1$ if packet i is an outgoing request and $i+1$ is an incoming response, providing high-level semantic supervision for multi-modal fusion.

Flow-level Statistical Reconstruction ($\mathcal{L}_{stat}$) regularizes the global representation $\mathbf{g}$ by recovering macroscopic flow characteristics. This loss combines a Kullback-Leibler (KL) divergence term for size distributions and MSE terms for direction ratios and IAT statistics:

$$\mathcal{L}_{stat} = D_{KL}(P_{size} \parallel \hat{P}_{size}) + \|\mathbf{r} - \hat{\mathbf{r}}\|^2 + \|\sigma_{iat} - \hat{\sigma}_{iat}\|^2, \quad (22)$$

where P_{size} is the empirical size histogram, $\mathbf{r}$ is the up/down packet ratio, and σ_{iat} denotes the mean and standard deviation of the flow’s IAT sequence. This ensures that the aggregated representation preserves global invariants while the individual heads specialize in packet-level details.

B Model Architecture and Hyperparameters

The detailed hyperparameter settings for both pre-training and fine-tuning phases are summarized in Tab. 6. The byte-level encoder utilizes a hidden dimension of 128 with 3 layers. The packet-level encoder employs a hidden dimension of 260 (to accommodate 5 distinct attention heads, each with dimension 52) with 4 layers. Both encoders use the GeLU activation function and Pre-LN normalization.

Table 6. Hyperparameter configurations for reproducibility.

Hyperparameter	Pre-training	Fine-tuning
Optimizer	AdamW	AdamW
Initial Learning Rate	1×10^{-4}	1×10^{-3}
Batch Size	128 (16×8 GPUs)	128
Training Epochs/Steps	160k steps	20 epochs
Warmup Steps/Epochs	2% total steps	10 epochs
Weight Decay	0.01	0.05
Dropout Rate	0.1	0.1
Mask Probability	0.25	N/A
LR Scheduler	Linear + Cosine	Linear + Cosine
SWA	N/A	Enabled (last 10 epochs)

Table 7. Summary of datasets.

Dataset	Usage	Flows	Task Type
MAWI / CIC-IoT	Pre-train	3.0M	Representation
CipherSpectrum	Downstream	120K	TLS 1.3 Identification
CSTNET-TLS 1.3	Downstream	46K	Real-world Robustness
DataCon2021	Downstream	1K	Proxy Detection

C Downstream Dataset Details

To ensure the robustness and generalizability of our proposed model, we employ a diverse collection of datasets spanning large-scale backbone traffic for pre-training and contemporary encrypted sessions for downstream tasks.

Pre-training Corpus. Our pre-training phase leverages the MAWI (specifically the 2024.10.19 trace) [13] and CIC-IoT-2022 [15] datasets to capture fundamental protocol structures across different network environments. The combined corpus comprises 68 GB of raw PCAP data, representing approximately 3M bidirectional flows. While MAWI provides high-diversity backbone traffic, CIC-IoT-2022 contributes traces from resource-constrained device environments, ensuring the model’s exposure to varied network conditions and traffic patterns.

CipherSpectrum Benchmark. The primary evaluation of TLS 1.3 classification performance is conducted on CipherSpectrum [58], a contemporary dataset designed to mitigate the systemic flaws of legacy benchmarks. CipherSpectrum consists of 120,000 sessions covering 40 distinct domains. Unlike older datasets such as ISCVPN2016 or USTC-TFC2016, which contain up to 98.9% unencrypted traffic and rely on deprecated ciphers like 3DES, CipherSpectrum exclusively features encrypted traffic using TLS 1.3 mandated suites: AES-128-GCM, AES-256-GCM, and CHACHA20-POLY1305. This ensures the model learns intrinsic application patterns rather than artifacts from unencrypted leakage or vulnerable encryption algorithms.

CSTNET-TLS 1.3. For web traffic identification, we employ the CSTNET-TLS 1.3 dataset [34]. Unlike earlier benchmarks, CSTNET-TLS 1.3 exclusively contains traffic encrypted with current, secure Authenticated Encryption with Associated Data (AEAD) ciphers.

DataCon2021. To assess the model’s behavior on operational monitoring workloads beyond ordinary web traffic, we include the DataCon2021 dataset [16]. This dataset provides real-world encrypted proxy traces with heterogeneous tunnel behaviors. It complements the web-traffic datasets by testing whether the classifier can separate more diverse encrypted tunnels under the same side-channel input constraints.

D Detailed Website/Service-Type Analysis

This section provides granular interpretations for the four recurrent head-preference topologies identified in the main paper. Our analysis aggregates Layer 4 gating weights per domain on the AES-128 dataset.

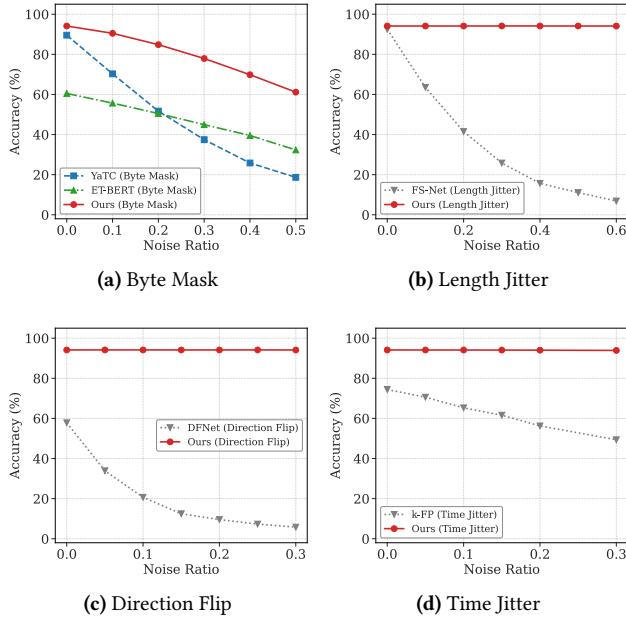


Figure 16. Robustness analysis. Comparison of classification accuracy between our method and baselines under four different traffic perturbation strategies.

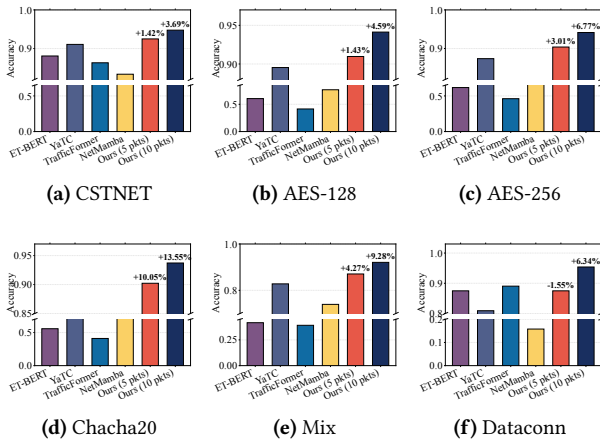


Figure 17. Fair comparison under a 5-packet constraint (same input budget as YaTC).

Mechanistic analysis of protocol abstraction. To elucidate the model’s internal abstraction mechanism, we analyze adaptive gating weights of samples randomly selected from the AES-128 dataset (using seed=42 for reproducibility). Fig. 18 and Fig. 19 show a transition from stable shallow fusion to traffic-dependent selection in deeper layers. Layer 1 assigns similar coefficients across samples, whereas Layer 4 strongly changes the active heads for different flows, indicating that Prometheus evolves from generic packet perception to selective protocol-behavior abstraction.

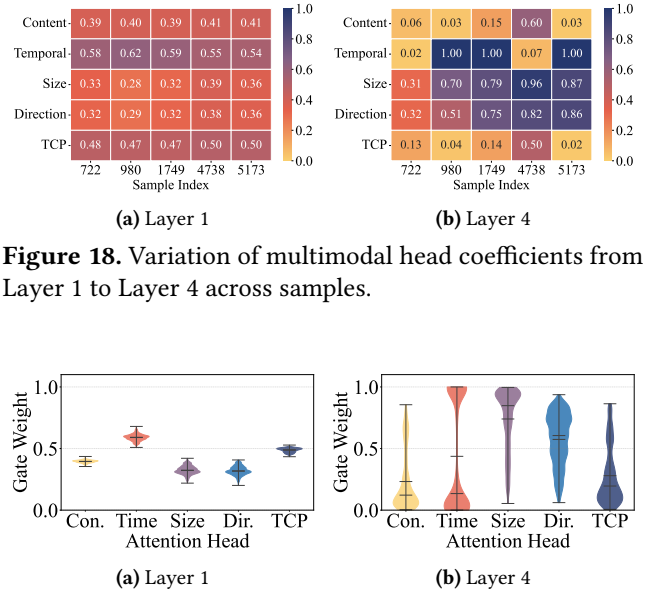


Figure 18. Variation of multimodal head coefficients from Layer 1 to Layer 4 across samples.

Figure 19. Distribution of adaptive gating weights across attention heads at a shallow layer (Layer 1) and a deep layer (Layer 4).

Periodic Automation: Automated periodicity and structural determinism. For services with pronounced periodic patterns (e.g., google.com, ubuntu.com), the model simultaneously activates Temporal and Size heads, forming a distinctive “fan-shaped” topology. This traffic typically originates from heartbeat mechanisms or automatic updates characterized by millisecond-level regularity and deterministic payload sizes (e.g., fixed JSON/XML formats). The model integrates these *temporal rhythms* and *stable patterns* to distinguish automated machine traffic from interactive human flows.

Static Retrieval: Robustness via noise rejection. In scenarios like steamstatic.com, the topology collapses into a single-axis structure dominated by the Size dimension. While temporal inter-arrival times (IATs) are often informative, they become high-entropy noise in bulk downloads due to network jitter and bandwidth contention. Conversely, file sizes remain invariant across network conditions. By suppressing Temporal weights and emphasizing Size, the model exhibits an explicit “denoising” strategy that favors robust physical attributes.

Complex Infrastructure: Residual protocol metadata. For infrastructure services such as cloudfront.net, the model increases Content and TCP-interaction weights, forming a “butterfly-shaped” topology. These services involve complex TLS negotiations or CDN scheduling, where residual metadata (e.g., handshake/transport field length patterns) provides richer fingerprints than coarse statistics. This indicates that even under encryption, the model exploits side-channel signals to distinguish complex service types.

Table 8. Five- versus ten-packet quality–collection-time trade-off. Collection time is the median first-to-kth-packet delay; quality is Heavy Model macro-F1 under matched 300-byte-per-packet runs.

Dataset	t_5 (ms)	t_{10} (ms)	F1@5	F1@10
CSTNET	7.1	69.0	0.930	0.939
AES-128	13.5	42.2	0.889	0.942
AES-256	13.6	41.0	0.884	0.941
ChaCha20	15.2	44.5	0.877	0.936
Mixed	14.9	42.6	0.837	0.922

Sparse Micro-interactions: Feature sparsity as a fingerprint. For micro-interaction and tracking services (e.g., *hsadspixel.net*), we observe a “low-activation” regime where all heads maintain small weights. These extremely short, one-off requests lack stable temporal rhythms or size patterns. Rather than forcing a fit to irrelevant dimensions, the model treats the *absence of salient structure* itself as a discriminative behavioral fingerprint.

Across these categories, the model consistently exhibits *context-aware feature selection*: leveraging size invariance for static delivery, temporal regularity for automation, residual metadata for complex infrastructures, and feature sparsity for micro-interactions. These emergent patterns confirm that the adaptive gating mechanism captures semantically meaningful protocol behaviors beyond dataset-specific statistical correlations.

E Supplementary Experiments

E.1 Packet-Budget Collection Time

Figure 20 plots the empirical distribution of the time from a flow’s first packet to its fifth and tenth packet. The four CipherSpectrum subsets show a consistent tens-of-milliseconds median increment when extending the budget, while CSTNET and DataCon have broader tails. The accompanying percentiles and validity counts are reported in the revision artifact; flows with fewer than ten packets are not assigned a fabricated tenth-packet time.

E.2 Full Classification Metrics

Tab. 9 provides the full metric breakdown corresponding to the compact macro-F1 summary in the main paper.

E.3 Per-Class Error Analysis

Table 10 expands the aggregate scores with the three lowest-F1 classes in each dataset. CipherSpectrum test classes are balanced, CSTNET uses anonymous numeric class IDs, and DataCon2021 labels identify proxy tools and direct browser traffic.

The errors exhibit distinct patterns across the three dataset families. In CipherSpectrum, Google Tag Manager is consistently confused with Google Analytics across all cipher variants. CSTNET’s lowest-F1 classes are identified only by

numeric labels and have lower support than its median class. In DataCon2021, WireGuard, OpenVPN-over-UDP, Psiphon, and Shadowsocks are classified correctly in the held-out test set; the remaining errors concentrate on the multi-protocol proxy clients Netch and V2Ray.

Case study: Google Tag Manager and Google Analytics. We analyze this recurring CipherSpectrum confusion using model-free features from the held-out flows. The error is asymmetric on the three homogeneous-cipher subsets: Tag Manager has high precision (0.966–1.000) but low recall (0.380–0.527), whereas Analytics has high recall (0.987–1.000) but lower precision (0.614–0.679).

No single coarse flow statistic consistently separates the pair. Their feature distributions differ more clearly in dispersion and directional proximity: on AES-128, Tag Manager has a mean within-class distance of 3.09, compared with 1.06 for Analytics, and 42.0% of Tag Manager flows are closest to the Analytics centroid among the other classes (Fig. 21). The same directional relationship appears across all four CipherSpectrum variants, with 35–42% attribution from Tag Manager to Analytics and none in the reverse direction. These observations are consistent with dispersed Tag Manager flows overlapping a more compact Analytics region, explaining the asymmetric classification errors.

E.4 Decision-Boundary Visualization

Fig. 22 visualizes decision-boundary refinement during the AES-128 $\Rightarrow$ ChaCha20 drift. Initial boundaries are misaligned with the shifted distribution, while online distillation reshapes the Light Model’s decision regions around the new encryption pattern.

E.5 Frozen Encoder Transferability

To isolate representation quality from fine-tuning capacity, we evaluate under frozen encoders in Fig. 23. Our method consistently outperforms baselines; notably, it maintains 67.19% accuracy on AES-128 where YaTC and TrafficFormer drop below 12%. This demonstrates that protocol-aware inductive biases enable inherently transferable representations.

E.6 Probing Task Evaluation

To verify whether our model captures meaningful protocol semantics rather than merely fitting dataset-specific statistics, we employ a *linear probing* evaluation across the AES-128, AES-256, and Mixed datasets. We freeze the pre-trained backbone and attach lightweight linear heads to perform four protocol-centric tasks: Direction Prediction, Inter-Arrival Time (IAT) Anomaly Detection, Packet Size Prediction, and TCP Request-Response Matching.

As shown in Tab. 11, our method demonstrates superior semantic understanding, particularly in the TCP Request-Response Matching task which requires reasoning about causality and interaction logic. While the baseline YaTC struggles significantly with a matching accuracy of only

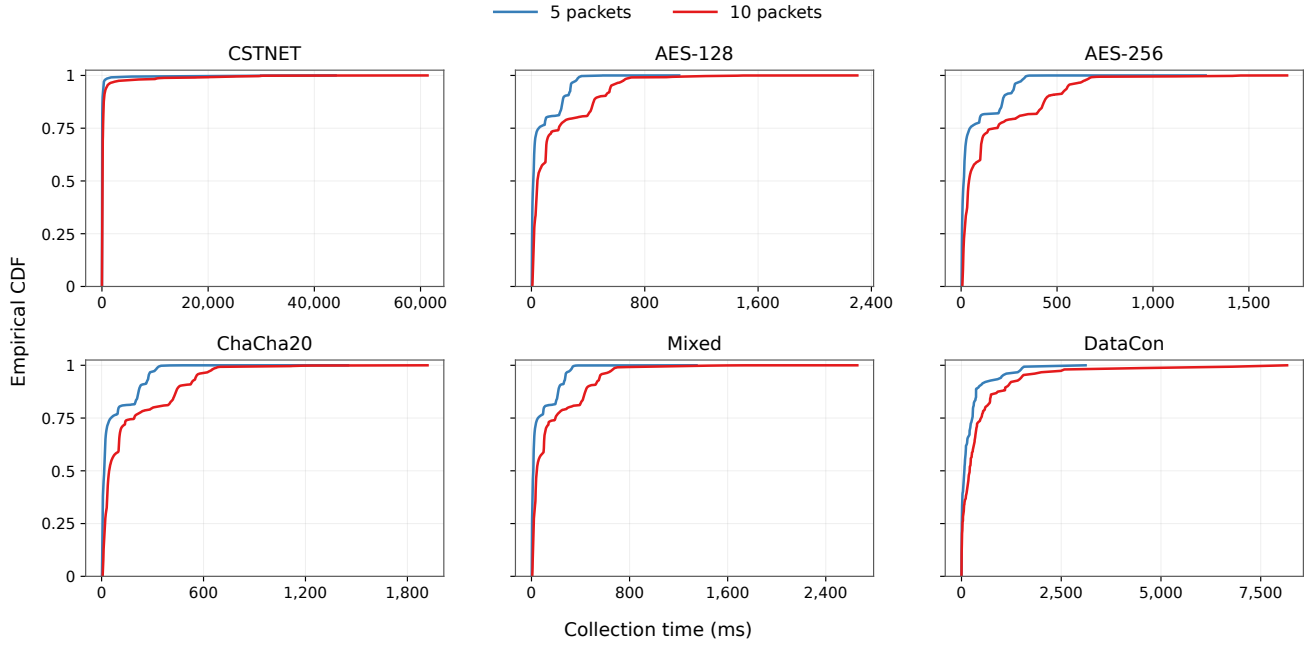


Figure 20. Empirical CDFs of first-packet-to-kth-packet collection time on the held-out test flows. The blue and red curves denote five- and ten-packet budgets, respectively; DataCon uses the available ten-packet trace for timing only.

Table 9. Full classification results comparing our Heavy Model against baseline methods on six datasets.

Dataset Method	CSTNET-TLS1.3				TLS-AES-128-GCM				TLS-AES-256-GCM			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
ET-BERT	0.8807	0.8698	0.8649	0.8651	0.6048	0.6135	0.6048	0.6040	0.6173	0.6313	0.6173	0.6155
YaTC	0.9107	0.9059	0.8987	0.8992	0.8954	0.9048	0.8953	0.8950	0.8738	0.8799	0.8738	0.8725
TrafficFormer	0.8631	0.8587	0.8517	0.8523	0.4145	0.4101	0.4145	0.4075	0.4611	0.4559	0.4611	0.4542
NetMamba	0.8335	0.8322	0.8062	0.8089	0.7668	0.8626	0.7668	0.8021	0.7421	0.7947	0.7421	0.7568
Ours	0.9476	0.9459	0.9366	0.9392	0.9413	0.9574	0.9413	0.9417	0.9415	0.9521	0.9415	0.9413

Dataset Method	TLS-CHACHA20POLY1305				TLS-MIX				DataCon2021			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
ET-BERT	0.5577	0.5723	0.5577	0.5591	0.4170	0.4269	0.4170	0.3279	0.8750	0.8993	0.8866	0.8749
YaTC	0.8017	0.8210	0.8017	0.8003	0.8279	0.8363	0.8279	0.8278	0.8092	0.8521	0.8254	0.8235
TrafficFormer	0.4106	0.4104	0.4106	0.4040	0.3917	0.3859	0.3917	0.3828	0.8905	0.9082	0.8914	0.8957
NetMamba	0.7206	0.7582	0.7206	0.7331	0.7394	0.7938	0.7394	0.7585	0.1579	0.0425	0.1437	0.0604
Ours	0.9372	0.9475	0.9372	0.9356	0.9207	0.9292	0.9207	0.9219	0.9539	0.9591	0.9580	0.9579

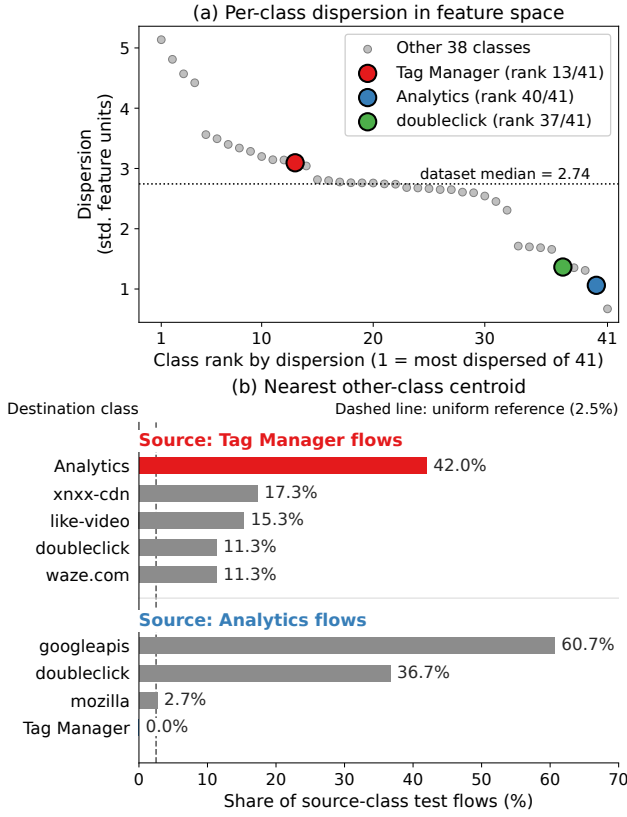
35.29% on the Mixed dataset, our model achieves 100%, indicating a robust grasp of duplex switching states. Similarly, in IAT Anomaly Detection, we observe a consistent performance advantage, outperforming YaTC by 4.31% in F1-score on the Mixed dataset. These results confirm that the pre-training objective successfully forces the model to learn high-level protocol state machines and interaction mechanics, distinct from low-level statistical correlations.

E.7 Robustness Against Traffic Perturbations

To assess the resilience of Prometheus, we evaluate its performance under four representative perturbations: payload/byte masking, packet length jitter, direction flipping, and time jitter. As illustrated in Fig. 16, Prometheus exhibits a more graceful degradation trend compared to prior methods. We attribute this stability to our protocol-guided inductive biases, which encourage the model to learn structural handshake patterns rather than over-fitting to volatile, low-level statistical features.

Table 10. Lowest-F1 Heavy Model classes on each held-out test set.

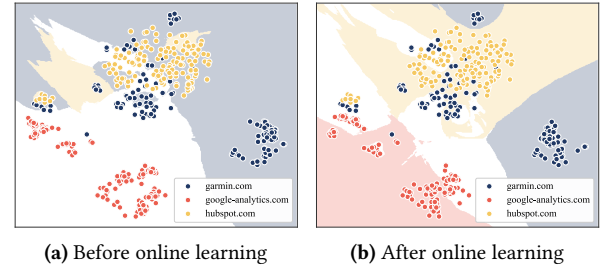
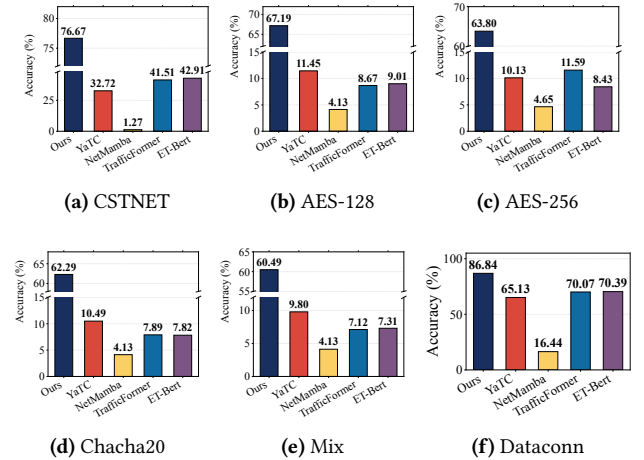
Dataset	Class	PR	RC	F1	N	Dataset	Class	PR	RC	F1	N
AES-128	googletagmanager.com	1.000	0.467	0.636	150	AES-256	googletagmanager.com	1.000	0.527	0.690	150
AES-128	trustarc.com	0.542	0.993	0.701	150	AES-256	flipboard.com	0.907	0.587	0.713	150
AES-128	google-analytics.com	0.652	1.000	0.789	150	AES-256	trustarc.com	0.601	0.993	0.749	150
ChaCha20	googletagmanager.com	0.966	0.380	0.545	150	Mixed	googletagmanager.com	0.744	0.612	0.671	152
ChaCha20	google-analytics.com	0.614	0.987	0.757	150	Mixed	trustarc.com	0.607	0.822	0.698	152
ChaCha20	trustarc.com	0.629	0.960	0.760	150	Mixed	hubspot.com	0.601	0.842	0.701	152
CSTNET	74	0.800	0.444	0.571	18	DataCon	10 (netch)	0.812	0.867	0.839	15
CSTNET	45	0.636	0.636	0.636	22	DataCon	2 (v2ray)	0.867	0.867	0.867	15
CSTNET	85	0.733	0.611	0.667	18	DataCon	6 (firefox)	1.000	0.867	0.929	15

**Figure 21.** Model-free analysis of the Tag Manager/Analytics confusion on AES-128. (a) Within-class dispersion in the standardized side-channel feature space. (b) Nearest other-class centroid attribution for held-out flows.

E.8 Fair Comparison under Strict Budgets

To ensure a fair evaluation against state-of-the-art methods like YaTC, we analyze a constrained scenario where Prometheus is limited to the first 5 packets of each flow.

As shown in Fig. 17, even under this strict input budget, Prometheus maintains a performance lead. This suggests that protocol-aware modeling improves **sample efficiency**, capturing essential classification cues earlier in the session. Furthermore, we observe that in TLS-heavy environments, extending the budget beyond 5 packets yields diminishing

**Figure 22.** Visualization of decision boundaries before and after online learning. Online distillation refines the Light Model's boundary around hard samples.**Figure 23.** Classification accuracy with frozen encoders for our Heavy Model compared with baselines.

but significant gains, as it allows the model to capture deeper application-layer interaction sequences that are often missed by truncated analysis.

F Details of Probing Task Designs

To strictly evaluate the semantic representation capabilities of our pre-trained model, we adopted a *Linear Probing* methodology. For all tasks described below, the parameters of the pre-trained backbone encoder are frozen. We only train

Table 11. Comparison of baselines on probing tasks (F1-Score).

Task	Dataset	TrafficFormer	ET-BERT	NetMamba	YaTC	Ours
Direction	AES-128	0.0455	0.3894	0.7032	0.9105	1.0000
	AES-256	0.0967	0.6201	0.0000	0.8922	0.9974
	Mixed	0.6620	0.6620	0.6710	0.9223	0.9959
IAT	AES-128	0.6717	0.6717	0.5008	0.6915	0.7160
	AES-256	0.6717	0.6717	0.5264	0.6915	0.7208
	Mixed	0.6734	0.6734	0.3249	0.6797	0.7228
Size Pred	AES-128	0.1406	0.1406	0.1406	0.3302	0.3716
	AES-256	0.1760	0.1282	0.1282	0.3349	0.3667
	Mixed	0.1655	0.1655	0.1900	0.4273	0.4677
TCP Match	AES-128	0.9848	0.6566	0.0000	0.5246	1.0000
	AES-256	0.9930	0.6543	0.0000	0.6833	1.0000
	Mixed	0.9885	0.6572	0.0000	0.3529	1.0000

lightweight task-specific heads (Multi-Layer Perceptrons) on top of the fixed flow representations.

F.1 Direction Prediction

This task assesses whether the model understands the duplex nature of network communication (i.e., the alternating pattern of incoming and outgoing packets). Regarding input construction, given a flow sequence of length L , we select a target position t and mask the feature vector of the packet at t with a learnable mask token, denoted as x_{masked} . The objective requires the model to predict the direction $d_t \in \{\text{IN}, \text{OUT}\}$ of the masked packet based solely on the surrounding bidirectional context, optimized via standard Cross-Entropy loss.

F.2 IAT Anomaly Detection

This task evaluates the model's sensitivity to fine-grained timing semantics and its ability to distinguish natural network jitter from artificial anomalies. For sample generation, we strictly followed a statistical injection strategy. Positive samples ($y = 1$) consist of original, unmodified packet sequences. For negative samples ($y = 0$), we randomly select a packet at index t and perturb its inter-arrival time (IAT). To ensure the anomalies are distinct yet challenging, perturbations are scaled based on the flow's statistical properties (mean μ and standard deviation σ of valid IATs). Anomalies include "spikes" ($5\mu \sim 10\mu$), "drops" ($0.001\mu \sim 0.01\mu$), or extreme outliers. The objective is a binary classification to determine if the sequence contains a manipulated timing artifact.

F.3 Packet Size Prediction

This task tests whether the model captures the logic of payload sizes (e.g., MTU limits, handshake packet sizes). We employ a **Discretization** strategy: instead of continuous regression, we formulate this as a classification problem where packet sizes are discretized into K bins (e.g., $[0, 40]$, $[40, 80]$, $\dots$, $[1200, 1500]$) to capture protocol-specific size distinctiveness. Similar to the Direction Prediction task, the **Objective**

involves masking the packet at index t and predicting the correct size bin $b_t \in \{1, \dots, K\}$.

F.4 TCP Request-Response Matching

This is the most complex task, designed to evaluate causal reasoning and the understanding of TCP state machines (e.g., SEQ/ACK numbers). We utilize a specific triplet construction method: for a given request packet p_{req} , we select the Ground Truth response p_{resp}^+ and a "Hard Negative" response p_{resp}^- . Critically, the Hard Negative is sampled from other response packets *within the same flow*. This constraint prevents the model from relying on easy flow-level statistics (like TTL or IP addresses) and forces it to learn packet-level interaction logic. The model input consists of the concatenated representations $[h_{\text{req}}; h_{\text{resp}}^+; h_{\text{resp}}^-]$, and the objective is to identify the correct causal response via binary classification.

G Extended Qualitative Comparison

Table 12 expands the compact comparison in the main paper with individual modeling approaches, protocol priors, and additional execution systems.

H Reproducibility and Artifacts

We provide a public artifact containing the implementation of the Heavy and Light models, the distillation pipeline, preprocessing scripts, and experiment configuration files at <https://github.com/D4wnnn/Prometheus-Traffic-Analysis>. The artifact also includes generation tools and processed feature files for the CipherSpectrum and CSTNET-TLS 1.3 evaluations, together with a README.md describing software dependencies, hardware requirements, and reproduction steps. The artifact is released under an open-source license.

Table 12. Qualitative comparison with closely related traffic-classification models and representative data-plane or switch-first systems. “No” indicates that no online model-update loop is reported in the cited work.

System	Task / scope	Representation	Execution	Protocol priors	Online adaptation
<i>Traffic-classification models</i>					
TrafficFormer [78]	Encrypted traffic classification	Pre-trained Transformer over traffic tokens	CPU/GPU	Generic attention	No
YaTC [75]	Encrypted traffic classification	Image-like packet representation with MAE	CPU/GPU	Implicit	No
NetMamba [55]	Encrypted traffic classification	State-space sequence model	CPU/GPU	Implicit	No
MM4Flow [68]	Versatile traffic analysis	Multi-modal pre-trained flow representation	CPU/GPU	Multi-modal fusion	No
tFusion [21]	Few-shot malicious-traffic detection	Cross-modality feature fusion	CPU/GPU	Cross-modal fusion	No
<i>Data-plane and switch-first systems</i>					
Akem et al. [1]	In-switch encrypted-traffic classification	Random forest over packet-size and arrival-time features	P4/Tofino switch	Feature-based	No
Taurus [50]	Line-rate per-packet ML	Pipelined per-packet MapReduce inference	Switch/NIC with SIMD extensions	Model-dependent	No
Brain-on-Switch [67]	Line-rate traffic analysis	Compact NN over packet length and delay	Switch-first; off-switch server	Feature-based	No
Quark [72]	Switch-native neural inference	CNN executed entirely on the switch	Programmable data plane	Architecture-dependent	No
Prometheus	Encrypted flow classification with deployment adaptation	Protocol-aware Heavy Model plus distilled Light Model	DPDK CPU + GPU backend	Temporal, size, direction, request-response	Asynchronous distillation